THÀNH ĐOÀN
TP. HỒ CHÍ MINH

ĐẠI HỌC QUỐC GIA
TP. HỒ CHÍ MINH

SỞ KHOA HỌC VÀ CÔNG NGHỆ
TP. HỒ CHÍ MINH

CÔNG TRÌNH DỰ THI
GIẢI THƯỞNG SINH VIÊN NGHIÊN CỨU KHOA HỌC EURÉKA
LẦN THỨ 28 NĂM 2026

TÊN CÔNG TRÌNH: THIẾT KẾ VÀ XÂY DỰNG DRONE SỬ DỤNG ESP32 VÀ MPU6050

- LĨNH VỰC NGHIÊN CỨU: Kỹ thuật công nghệ
- CHUYÊN NGÀNH: Điện, Điện tử

Mã số công trình: …….

MỤC LỤC

CHƯƠNG I. TỔNG QUAN ĐỀ TÀI VÀ CƠ SỞ LÝ THUYẾT 1
1.1. Lý do chọn đề tài 1
1.2. Mục tiêu của đề tài 1
1.3. Đối tượng và phạm vi nghiên cứu 2
1.4. Phương pháp thực hiện 2
1.5. Tổng quan về drone quadcopter 3
1.6. Nguyên lý tạo lực nâng 3
1.7. Chiều quay CW và CCW 4
1.8. Các chuyển động của drone 5
1.8.1. Góc Roll (Góc cuộn – Xoay quanh trục X) 5
1.8.2. Góc Pitch (Góc chúc/ngóc mũi – Xoay quanh trục Y) 5
1.8.3. Góc Yaw (Góc lệch/xoay hướng – Xoay quanh trục Z) 6
1.9. Vi điều khiển ESP32 6
1.10. Cảm biến MPU6050 8
1.11. Giao tiếp I²C 9
1.12. Điều chế độ rộng xung PWM 10
1.13. Động cơ Brushless A2212 2450KV và ESC 30A 11
1.14. Pin LiPo SHANG YI 3S 11,1 V – 2200 mAh và hệ thống nguồn 12
1.15. Tay cầm Microzone 6C Mini và mạch thu Microzone MC7RE-V2 13
1.16. Bộ lọc thông thấp và thuật toán Madgwick 14
1.17. Bộ điều khiển PID 15
CHƯƠNG II. PHÂN TÍCH, THIẾT KẾ HỆ THỐNG, PHẦN CỨNG VÀ PHẦN MỀM 16
2.1. Yêu cầu chức năng 16
2.2. Yêu cầu phi chức năng 16
2.3. Sơ đồ khối hệ thống 17
2.4. Nguyên lý hoạt động tổng thể 18
2.5. Thiết kế hệ thống điều khiển bằng tay cầm 19
2.6. Lựa chọn cấu hình phần cứng 20
2.7. Danh sách linh kiện 21
2.8. Sơ đồ kết nối ESP32 và MPU6050 21
2.9. Sơ đồ mạch điều khiển động cơ 21
2.10. Hệ thống nguồn 22

2.11. Khung và bố trí linh kiện 22
2.12. Sơ đồ đấu nối toàn hệ thống 23
2.13. Bảng chân kết nối 24
2.14. Môi trường phát triển 25
2.15. Cấu trúc chương trình 25
2.16. Khởi tạo MPU6050 26
2.17. Hiệu chuẩn cảm biến 26
2.18. Lọc dữ liệu và ước lượng tư thế bằng Madgwick 27
2.19. Khởi tạo bộ thu tín hiệu (Rx) 28
2.20. Đọc và xử lý tín hiệu tay cầm 28
2.21. Chuyển đổi tín hiệu điều khiển 29
2.22. Điều khiển tốc độ động cơ 29
2.23. Thuật toán PID 30
2.24. Motor mixer 31
2.25. Lưu đồ chương trình 31
CHƯƠNG III. QUÁ TRÌNH LẮP RÁP, THỰC HIỆN, KIỂM THỬ VÀ ĐÁNH GIÁ 34
3.1. Thử từng động cơ 34
3.2. Lắp bốn động cơ 34
3.3. Lắp mạch và pin 35
3.4. Kiểm tra trọng tâm và chiều cánh 36
3.5. Những khó khăn trong quá trình thực hiện 36
3.6. Kế hoạch kiểm thử 36
3.7. Kết quả đọc MPU6050 37
3.8. Kết quả bay thử 39
3.9. Đánh giá tổng thể 40
CHƯƠNG IV. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 42
4.1. Kết luận 42
4.2. Hướng phát triển 42
TÀI LIỆU THAM KHẢO 44

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải thích
ESP32	Vi điều khiển trung tâm dùng để đọc cảm biến, nhận tín hiệu điều khiển và xuất tín hiệu đến các ESC.
IMU	Inertial Measurement Unit – Đơn vị đo lường quán tính, cảm biến đo gia tốc và vận tốc góc.
MPU	Motion Processing Unit – Đơn vị xử lý chuyển động, tên gọi chung của dòng cảm biến IMU do InvenSense sản xuất, ví dụ MPU6050.
I2C	Inter-Integrated Circuit – Chuẩn giao tiếp nối tiếp hai dây dùng để kết nối vi điều khiển với thiết bị ngoại vi.
PWM	Pulse Width Modulation – Điều chế độ rộng xung, tín hiệu được ESP32 sử dụng để điều khiển ESC.
PID	Proportional – Integral – Derivative – Bộ điều khiển gồm ba thành phần tỉ lệ, tích phân và vi phân.
CW	Clockwise – Chiều quay thuận theo chiều kim đồng hồ.
CCW	Counter-Clockwise – Chiều quay ngược chiều kim đồng hồ.
ESC	Electronic Speed Controller – Bộ điều tốc điện tử dùng để điều khiển động cơ brushless.
Tx	Transmitter – Tay cầm phát tín hiệu điều khiển.
Rx	Receiver – Mạch thu tín hiệu điều khiển trên drone.
RF	Radio Frequency – Tín hiệu vô tuyến dùng để truyền lệnh giữa tay cầm Tx và mạch thu Rx.
LiPo	Lithium Polymer – Loại pin sạc phổ biến trong các thiết bị bay.
GPIO	General Purpose Input/Output – Chân vào/ra đa dụng trên vi điều khiển.
LPF	Low-Pass Filter – Bộ lọc thông thấp dùng để giảm nhiễu tần số cao trong dữ liệu Gyro/Accel.
Madgwick	Thuật toán hợp nhất dữ liệu gia tốc kế và con quay hồi chuyển. Trong mã nguồn này dùng biến thể 6DOF vì MPU6050 không có từ kế.
ISR	Interrupt Service Routine – Hàm phục vụ ngắt dùng để đo độ rộng xung PWM từ mạch thu.
6DOF	Six Degrees of Freedom – Ước lượng tư thế từ gia tốc kế 3 trục và con quay hồi chuyển 3 trục.
ESP32Servo	Thư viện tạo tín hiệu PWM kiểu servo trên ESP32; trong hệ thống được dùng để phát xung 1000–2000 µs đến ESC.

CHƯƠNG I. TỔNG QUAN ĐỀ TÀI VÀ CƠ SỞ LÝ THUYẾT

1.1. Lý do chọn đề tài

Trong những năm gần đây, drone (thiết bị bay không người lái) ngày càng được ứng dụng rộng rãi trong nhiều lĩnh vực như chụp ảnh và quay phim trên không, khảo sát địa hình, nông nghiệp chính xác, kiểm tra công trình và cứu hộ cứu nạn. Sự phát triển của drone gắn liền chặt chẽ với sự tiến bộ của các hệ thống nhúng, khi các thiết bị bay hiện đại đều dựa trên một bộ xử lý trung tâm để thu thập dữ liệu cảm biến, tính toán thuật toán điều khiển và xuất tín hiệu điều khiển động cơ trong thời gian thực.

Hệ thống nhúng đóng vai trò then chốt trong một drone, bởi toàn bộ quá trình đọc cảm biến, xử lý dữ liệu, thực thi thuật toán cân bằng và giao tiếp với người điều khiển đều phải được thực hiện trên một vi điều khiển có tài nguyên giới hạn nhưng đòi hỏi tốc độ phản hồi cao. Việc nghiên cứu, thiết kế và xây dựng một mô hình drone quy mô nhỏ là cơ hội thực tế để sinh viên áp dụng kiến thức về vi điều khiển, cảm biến, điều khiển tự động và lập trình nhúng vào một hệ thống hoàn chỉnh.

Trong số các nền tảng vi điều khiển hiện có, ESP32 được nhóm lựa chọn nhờ tốc độ xử lý phù hợp, có nhiều chân GPIO, hỗ trợ PWM phần cứng và giao tiếp I^2C, đồng thời có cộng đồng phát triển lớn với nhiều tài liệu tham khảo.

1.2. Mục tiêu của đề tài

Mục tiêu tổng quát của đồ án là nghiên cứu, thiết kế và từng bước xây dựng một mô hình drone bốn động cơ có khả năng nhận lệnh từ tay cầm điều khiển thông qua mạch thu sóng vô tuyến, đồng thời bước đầu tìm hiểu và áp dụng thuật toán cân bằng dựa trên dữ liệu từ cảm biến MPU6050.

Các mục tiêu cụ thể mà nhóm hướng tới bao gồm:

- Xây dựng mô hình cơ khí drone bốn động cơ (quadcopter) với cấu hình phần cứng phù hợp với điều kiện thực tế của nhóm.
- Đọc và xử lý dữ liệu từ cảm biến MPU6050 để xác định góc nghiêng của drone theo hai trục roll và pitch.
- Lập trình ESP32 để điều khiển tốc độ của bốn động cơ thông qua tín hiệu PWM.

- Xây dựng cơ chế nhận và xử lý các lệnh throttle, roll, pitch và yaw từ tay cầm thông qua mạch thu sóng vô tuyến (Receiver – Rx).
- Tích hợp tay cầm điều khiển (Transmitter – Tx) và mạch thu sóng vô tuyến (Receiver – Rx) để truyền lệnh điều khiển đến ESP32.
- Tìm hiểu và áp dụng bộ lọc thông thấp, thuật toán Madgwick và bộ điều khiển PID nhằm hỗ trợ ước lượng tư thế và cân bằng drone theo các trục roll, pitch và yaw.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đồ án là mô hình drone dạng quadcopter ở quy mô đồ án sinh viên, sử dụng vi điều khiển ESP32 làm bộ xử lý trung tâm và cảm biến MPU6050 làm cảm biến quán tính chính. Phạm vi nghiên cứu được giới hạn cụ thể như sau:

- Drone được điều khiển trong phạm vi gần bằng tay cầm Tx và mạch thu Rx; phạm vi hoạt động thực tế cần được xác định bằng quá trình kiểm thử.
- Đồ án không tập trung vào định vị GPS, camera truyền hình ảnh trực tiếp (FPV) hay các chế độ bay tự động (autopilot).
- Khung drone sử dụng loại có sẵn trên thị trường, không thiết kế và in 3D khung riêng.
- Mạch điều khiển động cơ trong phiên bản đầu sử dụng board mạch đục lỗ hoặc mạch lắp ráp thủ công, không thiết kế PCB chuyên dụng.
- Nội dung kỹ thuật tập trung vào bốn thành phần chính: vi điều khiển ESP32, cảm biến MPU6050, hệ thống động cơ brushless kết hợp ESC và cơ chế nhận tín hiệu từ tay cầm thông qua mạch thu Rx.

1.4. Phương pháp thực hiện

Đồ án được thực hiện theo quy trình gồm các bước tuần tự, có thể điều chỉnh linh hoạt tùy theo tiến độ thực tế:

- Bước 1: Tìm hiểu lý thuyết về drone, cảm biến IMU, vi điều khiển ESP32 và các thuật toán cân bằng liên quan.
- Bước 2: Lựa chọn phần cứng phù hợp với cấu hình hệ thống, thời gian thực hiện và khả năng tiếp cận linh kiện của nhóm.
- Bước 3: Thiết kế sơ đồ khối tổng thể của hệ thống, xác định luồng dữ liệu giữa các thành phần.

- Bước 4: Lập trình từng chức năng riêng lẻ theo hướng module hóa: đọc cảm biến, đọc tín hiệu từ mạch thu Rx, xử lý lệnh điều khiển và điều khiển động cơ.
- Bước 5: Kiểm tra hoạt động của cảm biến MPU6050 độc lập trước khi tích hợp vào hệ thống.
- Bước 6: Kiểm tra giao tiếp giữa tay cầm Tx, mạch thu Rx và ESP32, bảo đảm các kênh tín hiệu thay đổi đúng theo thao tác điều khiển.
- Bước 7: Kiểm tra từng động cơ riêng lẻ để xác nhận chiều quay và đáp ứng tốc độ trước khi lắp đặt hoàn chỉnh.
- Bước 8: Tích hợp toàn bộ các module thành hệ thống hoàn chỉnh.
- Bước 9: Thử nghiệm hệ thống theo kế hoạch kiểm thử và đánh giá kết quả đạt được.

1.5. Tổng quan về drone quadcopter

Drone, hay còn gọi là thiết bị bay không người lái (Unmanned Aerial Vehicle – UAV), là một loại phương tiện bay được điều khiển từ xa hoặc hoạt động tự động mà không cần phi công ngồi trên khoang. Drone có nhiều dạng khác nhau như dạng cánh cố định (fixed-wing) hoặc dạng nhiều cánh quạt (multirotor), trong đó quadcopter là dạng drone sử dụng bốn động cơ gắn cánh quạt để tạo lực nâng và điều khiển chuyển động.

So với các dạng multirotor khác như tricopter (ba động cơ) hay hexacopter (sáu động cơ), quadcopter được sử dụng phổ biến nhất nhờ cấu trúc cơ khí đơn giản, số lượng động cơ vừa đủ để thực hiện đầy đủ các chuyển động cơ bản, đồng thời thuật toán điều khiển cũng tương đối dễ tiếp cận đối với người mới bắt đầu tìm hiểu về drone. Bốn động cơ được bố trí đối xứng theo hình chữ X hoặc hình chữ thập (+), giúp việc tính toán và cân chỉnh lực đẩy giữa các động cơ trở nên thuận tiện hơn.

Ưu điểm chính của quadcopter là khả năng cất cánh và hạ cánh thẳng đứng mà không cần đường băng, khả năng bay đứng yên tại một vị trí (hover) và tính linh hoạt trong việc thay đổi hướng bay. Tuy nhiên, quadcopter cũng có một số hạn chế như thời gian bay bị giới hạn bởi dung lượng pin, khả năng chịu tải thấp hơn so với các loại máy bay cánh cố định có cùng công suất, và yêu cầu thuật toán cân bằng liên tục để duy trì ổn định trong không trung.

1.6. Nguyên lý tạo lực nâng

Mỗi động cơ trên drone được gắn với một cánh quạt (propeller). Khi động cơ quay, cánh quạt đẩy không khí xuống phía dưới, theo định luật III Newton, không khí sẽ tạo ra một phản lực đẩy cánh quạt (và toàn bộ drone) theo chiều ngược lại, tức là hướng lên trên. Lực này được gọi là lực đẩy (thrust) của động cơ.

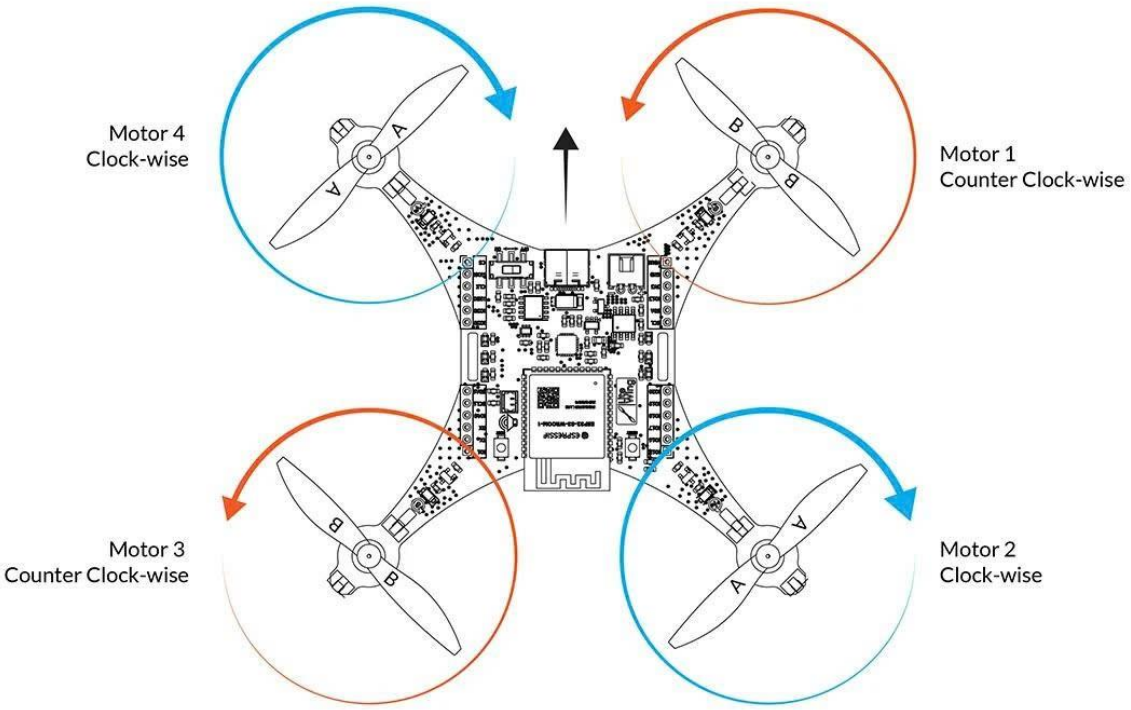
Tổng lực đẩy được tạo ra bởi bốn động cơ sẽ được so sánh với trọng lượng của toàn bộ drone để xác định trạng thái chuyển động theo phương thẳng đứng. Khi tổng lực đẩy bằng trọng lượng của drone, drone sẽ giữ nguyên độ cao (trạng thái hover). Khi tổng lực đẩy lớn hơn trọng lượng, drone sẽ được nâng lên; ngược lại, khi tổng lực đẩy nhỏ hơn trọng lượng, drone sẽ hạ xuống.

Việc điều chỉnh tổng lực đẩy được thực hiện thông qua việc tăng hoặc giảm đồng thời tốc độ của cả bốn động cơ, tương ứng với tín hiệu throttle mà người điều khiển gửi đến drone. Đồ án này không đi sâu vào các mô hình khí động học nâng cao như hiệu ứng dòng xoáy đầu cánh hay ảnh hưởng của mật độ không khí, mà chỉ tập trung vào nguyên lý cơ bản đủ để giải thích cách drone cất cánh, giữ độ cao và hạ cánh.

1.7. Chiều quay CW và CCW

Trên một drone quadcopter tiêu chuẩn, bốn động cơ không quay cùng một chiều mà được chia thành hai cặp: hai động cơ quay theo chiều kim đồng hồ (Clockwise – CW) và hai động cơ quay ngược chiều kim đồng hồ (Counter-Clockwise – CCW). Hai động cơ CW được bố trí chéo nhau, và tương tự hai động cơ CCW cũng được bố trí chéo nhau ở hai góc còn lại.

Nguyên nhân của cách bố trí xen kẽ này xuất phát từ định luật bảo toàn mô-men động lượng. Khi một động cơ quay theo một chiều nhất định để tạo lực đẩy, bản thân khung drone sẽ chịu tác động của một mô-men xoắn phản lực theo chiều ngược lại. Nếu cả bốn động cơ đều quay cùng một chiều, các mô-men xoắn phản lực sẽ cộng dồn lại và làm cho toàn bộ drone bị xoay quanh trục thẳng đứng một cách không mong muốn. Bằng cách bố trí hai động cơ quay CW và hai động cơ quay CCW xen kẽ nhau, các mô-men xoắn phản lực sẽ triệt tiêu lẫn nhau khi tốc độ của bốn động cơ bằng nhau, giúp drone giữ được hướng ổn định.



Hình 1: Sơ đồ bố trí bốn động cơ theo chiều quay CW và CCW

1.8. Các chuyển động của drone

Một drone quadcopter thực hiện được bốn loại chuyển động cơ bản là throttle, roll, pitch và yaw, được tạo ra bằng cách thay đổi tốc độ tương đối giữa các động cơ. Việc hiểu rõ nguyên lý của từng chuyển động là cơ sở quan trọng để xây dựng thuật toán motor mixer ở Chương II.

1.8.1. Góc Roll (Góc cuộn – Xoay quanh trục X)

Roll là chuyển động xoay của Quadcopter quanh trục dọc (trục X), kéo dài từ phần đuôi đến phần mũi của thiết bị.

Nguyên lý hoạt động:
Chuyển động Roll làm cho mặt phẳng của Quadcopter nghiêng sang trái hoặc sang phải. Để tạo chuyển động này, bộ điều khiển bay (Flight Controller) điều chỉnh sự chênh lệch lực đẩy (Thrust) giữa cặp động cơ bên trái và cặp động cơ bên phải.

Tác động khí động học:
Khi Quadcopter nghiêng theo trục Roll, một thành phần lực đẩy theo phương ngang được tạo ra, làm cho thiết bị dịch chuyển sang trái hoặc sang phải mà không làm thay đổi hướng của mũi máy bay.

1.8.2. Góc Pitch (Góc chúc/ngóc mũi – Xoay quanh trục Y)

Định nghĩa:
Pitch là chuyển động xoay quanh trục ngang (trục Y), đi qua trọng tâm của thân máy bay từ trái sang phải.

Nguyên lý hoạt động:
Chuyển động Pitch làm cho phần mũi của Quadcopter ngóc lên hoặc chúc xuống so với mặt phẳng ngang. Bộ điều khiển thực hiện điều này bằng cách tạo sự chênh lệch lực đẩy giữa cụm động cơ phía trước và cụm động cơ phía sau.

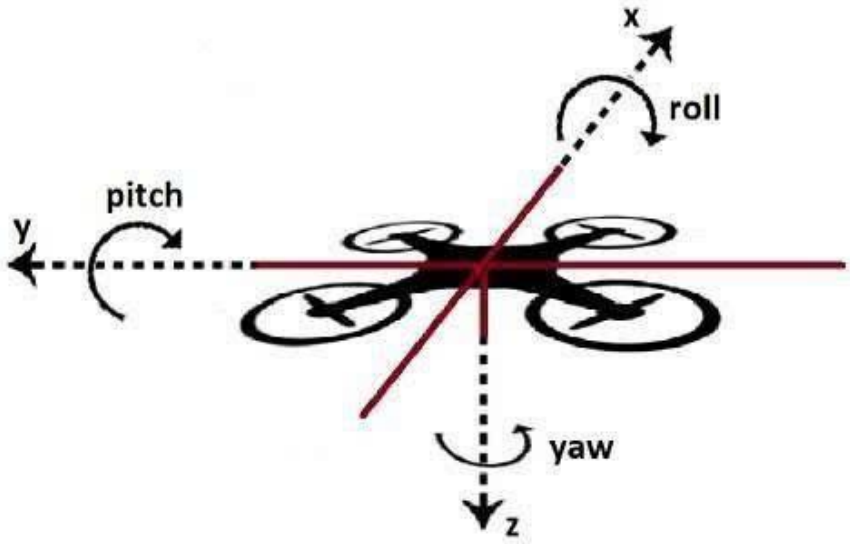
Tác động khí động học:
Khi góc Pitch thay đổi, vectơ lực đẩy của toàn bộ hệ thống sẽ nghiêng về phía trước hoặc phía sau, tạo thành phần lực theo phương ngang giúp Quadcopter di chuyển tiến (Forward) hoặc lùi (Backward).

1.8.3. Góc Yaw (Góc lệch/xoay hướng – Xoay quanh trục Z)

Định nghĩa:
Yaw là chuyển động xoay quanh trục thẳng đứng (trục Z), đi qua trọng tâm của Quadcopter theo phương từ trên xuống dưới.

Nguyên lý hoạt động:
Khác với Roll và Pitch, chuyển động Yaw không làm nghiêng mặt phẳng của Quadcopter mà chỉ làm mũi máy bay xoay sang trái hoặc sang phải. Chuyển động này được tạo ra nhờ sự chênh lệch mô-men xoắn phản lực (Reactive Torque), thay vì sự chênh lệch lực đẩy. Cụ thể, Quadcopter sẽ tăng tốc độ của cặp động cơ quay cùng chiều kim đồng hồ (CW) và giảm tốc độ của cặp động cơ quay ngược chiều kim đồng hồ (CCW), hoặc ngược lại, để tạo mô-men làm xoay thân máy bay.

Tác động khí động học:
Trục Yaw giữ vai trò quyết định trong việc thay đổi hướng bay và định hướng mũi của Quadcopter trong không gian.



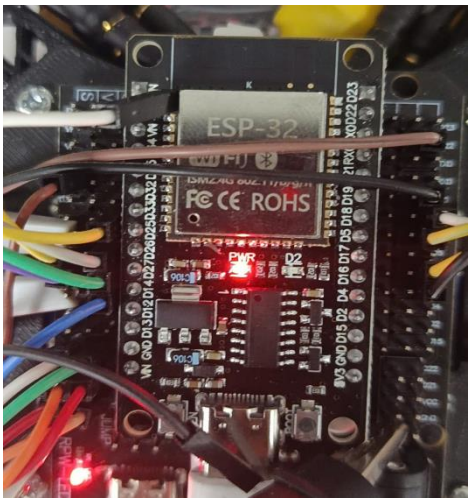
Hình 2: Minh họa bốn chuyển động cơ bản của drone: throttle, roll, pitch, yaw

1.9. Vi điều khiển ESP32

ESP32 là một dòng vi điều khiển do công ty Espressif Systems phát triển, được sử dụng rộng rãi trong các hệ thống nhúng nhờ khả năng xử lý, số lượng chân GPIO và các ngoại vi tích hợp.

Về mặt ngoại vi, ESP32 cung cấp nhiều chân GPIO có thể được cấu hình linh hoạt, bao gồm giao tiếp I²C để kết nối với MPU6050, các kênh PWM phần cứng để điều khiển bốn ESC và các chân đầu vào để nhận tín hiệu từ mạch thu Rx.

Trong hệ thống, ESP32 đọc MPU6050 qua I²C, đo sáu kênh PWM từ Rx bằng ngắt, chạy lọc thông thấp, Madgwick6DOF, PID và mixer, sau đó dùng ESP32Servo phát tín hiệu đến bốn ESC.



Hình 3: Bo mạch ESP32 thực tế được nhóm sử dụng trong đồ án

Hệ thống sử dụng các chân GPIO của ESP32 để kết nối với ESC, cảm biến MPU6050 và bộ thu tín hiệu (RX). Cụ thể như sau:

1.9.1. Chân tín hiệu điều khiển ESC theo mã nguồn

- ESC 1: GPIO26
- ESC 2: GPIO33
- ESC 3: GPIO14
- ESC 4: GPIO27

1.9.2. Chân kết nối bộ thu tín hiệu (RX)

- CH1 (Throttle): GPIO35
- CH2 (AIL/Roll): GPIO18
- CH3 (ELE/Pitch): GPIO17

- CH4 (RUD/Yaw): GPIO19
- CH5 (Throttle Cut): GPIO16
- CH6 (Aux): GPIO32

1.9.3. Chân giao tiếp với cảm biến MPU6050 (I²C)

- SCL: GPIO22
- SDA: GPIO21

1.9.4. Chân cấp nguồn

- 3.3 V: 01 chân cấp nguồn cho cảm biến MPU6050.
- 5 V: 02 chân cấp nguồn, gồm:
- 01 chân cấp nguồn từ mạch giảm áp (Buck Converter).
- 01 chân cấp nguồn cho bộ thu tín hiệu RX.

1.9.5. Chân nối đất (GND)

Tổng cộng sử dụng 07 chân GND, bao gồm:

- 04 chân GND cho bốn ESC.
- 01 chân GND cho cảm biến MPU6050.
- 01 chân GND cho bộ thu tín hiệu RX.
- 01 chân GND cho mạch giảm áp.

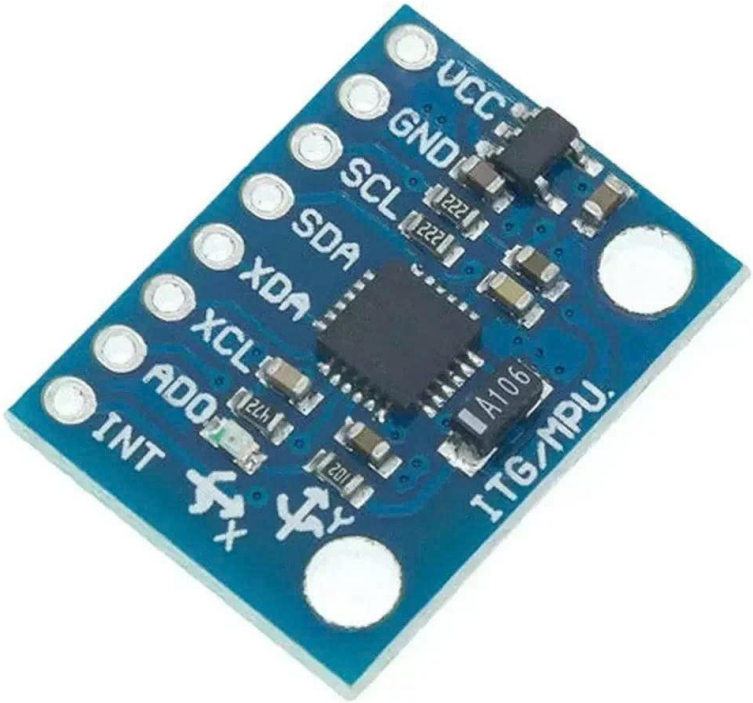
1.10. Cảm biến MPU6050

MPU6050 là một cảm biến IMU (Inertial Measurement Unit) 6 trục do hãng InvenSense sản xuất, tích hợp trong cùng một chip hai loại cảm biến: gia tốc kế ba trục (accelerometer) và con quay hồi chuyển ba trục (gyroscope). Nhờ tích hợp cả hai loại cảm biến trên cùng một chip, MPU6050 giúp giảm sai số do lệch trục cơ khí giữa hai cảm biến riêng biệt, đồng thời đơn giản hóa việc kết nối với vi điều khiển.

Gia tốc kế ba trục đo gia tốc theo ba phương X, Y, Z, bao gồm cả gia tốc trọng trường, cho phép xác định hướng nghiêng của cảm biến so với phương thẳng đứng khi ở trạng thái tĩnh hoặc chuyển động chậm. Con quay hồi chuyển ba trục đo vận tốc góc quay quanh ba trục X, Y, Z, cho phép xác định tốc độ thay đổi góc nghiêng theo thời gian.

MPU6050 giao tiếp với vi điều khiển thông qua chuẩn I²C, và dữ liệu đầu ra bao gồm sáu giá trị: gia tốc theo ba trục (Ax, Ay, Az) và vận tốc góc theo ba trục (Gx, Gy, Gz). Trong đồ án này, dữ liệu từ MPU6050 được lọc thông thấp và đưa vào thuật toán

Madgwick để ước lượng các góc roll, pitch và yaw, sau đó làm tín hiệu phản hồi cho bộ điều khiển PID.



Hình 4:Module cảm biến MPU6050

1.11. Giao tiếp I²C

I²C (Inter-Integrated Circuit) là một chuẩn giao tiếp nối tiếp đồng bộ, sử dụng hai đường tín hiệu là SDA (Serial Data – đường truyền dữ liệu) và SCL (Serial Clock – đường tín hiệu xung nhịp). Chuẩn I²C hoạt động theo mô hình master–slave, trong đó một thiết bị đóng vai trò master (thường là vi điều khiển) có thể giao tiếp với một hoặc nhiều thiết bị slave (như các module cảm biến) trên cùng một bus tín hiệu.

Mỗi thiết bị slave trên bus I²C được nhận diện thông qua một địa chỉ riêng (thường dài 7 bit). Đối với MPU6050, địa chỉ I²C mặc định thường là 0x68, có thể thay đổi thành 0x69 tùy theo mức điện áp được cấp cho chân AD0 của module.

Ưu điểm của chuẩn I²C là chỉ cần hai đường dây tín hiệu để kết nối nhiều thiết bị khác nhau, giúp đơn giản hóa việc đi dây so với các chuẩn giao tiếp cần nhiều đường tín hiệu riêng biệt cho mỗi thiết bị. Trong đồ án này, ESP32 đóng vai trò master, giao tiếp với MPU6050 (đóng vai trò slave) thông qua hai chân GPIO được cấu hình làm SDA và

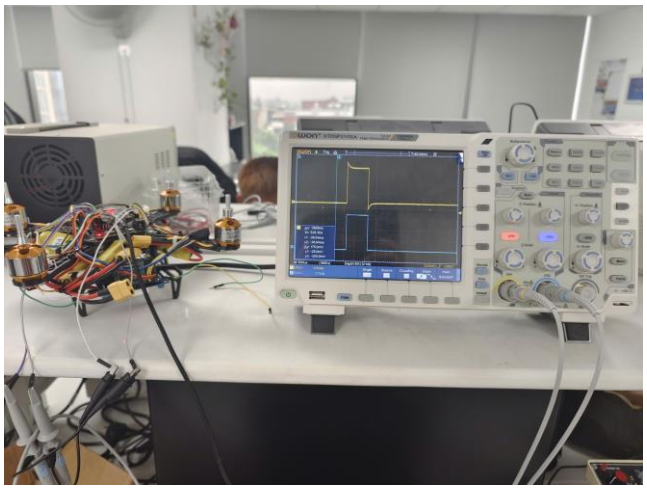
SCL, sử dụng thư viện I²C có sẵn trong môi trường lập trình để gửi lệnh cấu hình và đọc dữ liệu cảm biến.

1.12. Điều chế độ rộng xung PWM

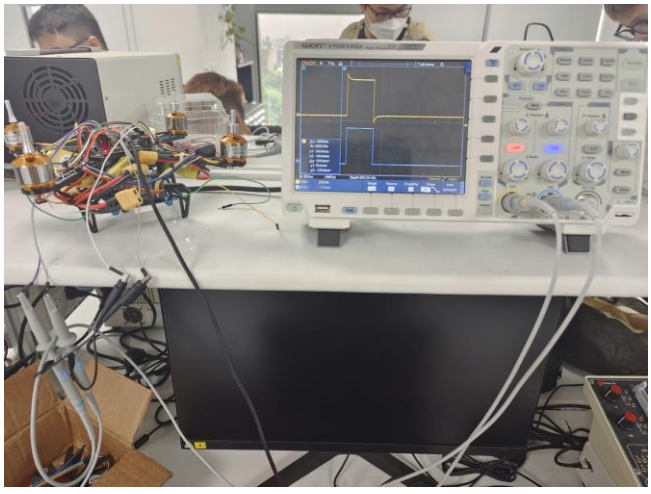
PWM (Pulse Width Modulation – điều chế độ rộng xung) là một kỹ thuật điều khiển công suất trung bình cấp cho một tải bằng cách thay đổi độ rộng của xung vuông trong một chu kỳ cố định, thay vì thay đổi biên độ điện áp. Một tín hiệu PWM được đặc trưng bởi hai thông số chính: chu kỳ (period), là khoảng thời gian lặp lại của tín hiệu, và duty cycle (chu kỳ nhiệm vụ), là tỉ lệ phần trăm thời gian tín hiệu ở mức cao trong một chu kỳ.

Duty cycle càng lớn, thời gian tín hiệu ở mức cao trong mỗi chu kỳ càng dài, dẫn đến công suất trung bình cấp cho tải càng lớn. Trong hệ thống của đồ án, ESP32 quy đổi giá trị điều khiển thành xung PWM có độ rộng trong khoảng 1000–2000 µs để gửi đến bốn ESC của các động cơ brushless.

Trong mã nguồn của nhóm, tín hiệu ESC không được tạo trực tiếp bằng LEDC mà thông qua thư viện ESP32Servo. Bốn đối tượng Servo được gắn với khoảng xung 1000–2000 µs; giá trị lệnh 0–180 được thư viện chuyển thành độ rộng xung tương ứng.

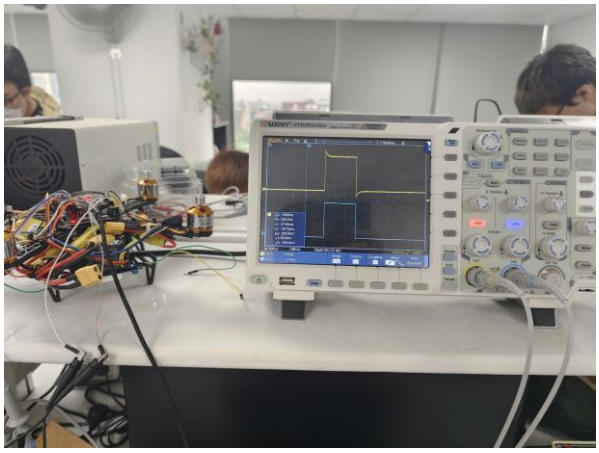


1000

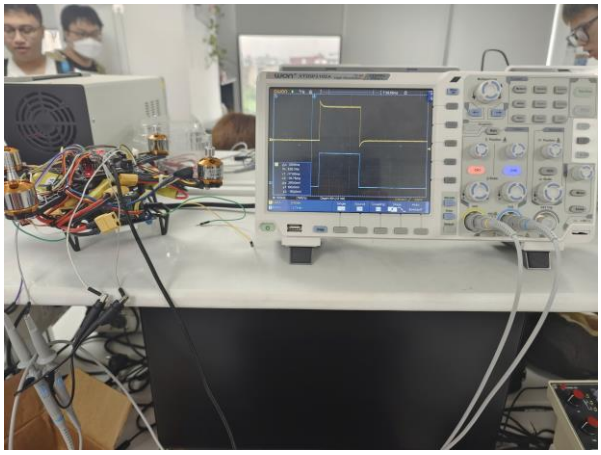


1250

Hình 5:Xung PWM ở mức 1000 và 1250



1500



2000

Hình 6 :Xung PWM ở mức 1500 và 2000

1.13. Động cơ Brushless A2212 2450KV và ESC 30A

Hệ thống sử dụng bốn động cơ Brushless A2212 2450KV, mỗi động cơ được điều khiển bởi một mạch điều tốc ESC 30A và lắp với cánh quạt 5 inch 5045.

Motor brushless là động cơ điện không sử dụng chổi than, trong đó việc chuyển mạch dòng điện trong các cuộn dây được thực hiện bằng mạch điện tử.

Động cơ được nhóm sử dụng là A2212 2450KV; chỉ số 2450KV biểu thị tốc độ quay không tải lý thuyết trên mỗi vôn điện áp cấp cho động cơ.

Mỗi động cơ được kết nối với một ESC 30A; ESC nhận tín hiệu điều khiển từ ESP32 và điều phối dòng điện ba pha để thay đổi tốc độ quay của động cơ.

Nguồn công suất của bốn ESC được lấy từ pin LiPo SHANG YI 3S 11,1 V – 2200 mAh, còn cánh quạt được sử dụng là loại 5 inch 5045 và phải được lắp đúng chiều CW hoặc CCW.



Hình 7: Động cơ brushless A2212 2450KV

1.14. Pin LiPo SHANG YI 3S 11,1 V – 2200 mAh và hệ thống nguồn

Pin được sử dụng trong hệ thống là pin LiPo SHANG YI cấu hình 3S, có điện áp danh định 11,1 V và dung lượng 2200 mAh. Pin gồm ba cell LiPo mắc nối tiếp; điện áp khi sạc đầy có thể đạt khoảng 12,6 V.

Dung lượng 2200 mAh thể hiện khả năng lưu trữ điện tích của pin và ảnh hưởng đến thời gian hoạt động của drone. Đầu nguồn chính sử dụng giắc XT60 để kết nối với đường cấp công suất cho bốn ESC 30A. Pin còn có giắc cân bằng dùng trong quá trình sạc cân bằng ba cell.

Pin LiPo cần được sử dụng và sạc bằng bộ sạc chuyên dụng hỗ trợ pin 3S. Không để pin bị đoản mạch, va đập mạnh, sạc quá mức hoặc xả quá sâu nhằm hạn chế nguy cơ phồng pin, hư hỏng hoặc cháy nổ.

Trong hệ thống của đồ án, pin LiPo SHANG YI 3S 11,1 V – 2200 mAh cấp nguồn công suất cho bốn ESC. Một nhánh nguồn được đưa qua mạch giảm áp DC-DC LM2596 để tạo điện áp 5 V cấp cho ESP32 và mạch thu Rx; cảm biến MPU6050 được cấp nguồn 3,3 V từ ESP32.



Hình 8: Pin LiPo SHANG YI 3S 11,1 V – 2200 mAh

1.15. Tay cầm Microzone 6C Mini và mạch thu Microzone MC7RE-V2

Hệ thống sử dụng tay cầm Microzone 6C Mini hoạt động ở băng tần 2,4 GHz, có 6 kênh điều khiển. Mạch thu Microzone MC7RE-V2 xuất sáu kênh xung PWM riêng biệt để ESP32 đo độ rộng xung và xác định lệnh điều khiển.

Theo mã nguồn, sáu kênh được nối như sau: CH1–Throttle vào GPIO35, CH2–AIL/Roll vào GPIO18, CH3–ELE/Pitch vào GPIO17, CH4–RUD/Yaw vào GPIO19, CH5–Gear/Throttle Cut vào GPIO16 và CH6–Aux vào GPIO32.

ESP32 đo xung PWM bằng ngắt theo cả cạnh lên và cạnh xuống, sau đó lọc các kênh điều khiển chính bằng bộ lọc thông thấp bậc một với hệ số b = 0,7. Các xung được chuẩn hóa thành throttle, roll, pitch và yaw trước khi đưa vào bộ điều khiển.



Hình 9: Tay cầm Microzone 6C

1.16. Bộ lọc thông thấp và thuật toán Madgwick

Dữ liệu gia tốc và vận tốc góc từ MPU6050 được hiệu chỉnh offset rồi lọc bằng bộ lọc thông thấp bậc một. Mã nguồn sử dụng B_accel = 0,14 cho gia tốc kế và B_gyro = 0,10 cho con quay hồi chuyển; các tham số được đặt cho vòng lặp mục tiêu 2000 Hz.

Do MPU6050 chỉ có gia tốc kế và con quay hồi chuyển, chương trình tự động gọi thuật toán Madgwick 6DOF với tham số B_madgwick = 0,04. Thuật toán cập nhật quaternion và chuyển đổi thành roll, pitch, yaw; trong đó yaw chỉ được suy ra từ gyro nên có thể bị trôi theo thời gian vì không có từ kế.

Tín hiệu tay cầm được chuẩn hóa thành throttle trong khoảng 0–1, góc đặt roll và pitch trong khoảng ±30°, và tốc độ yaw mong muốn trong khoảng ±160°/s. Roll và pitch được điều khiển theo góc, còn yaw được điều khiển theo tốc độ góc GyroZ.

Đầu ra PID được đưa vào bộ trộn tín hiệu cho cấu hình quadcopter chữ X. Bốn giá trị sau mixer được quy đổi sang miền 0–180 của thư viện ESP32Servo; khi gắn mỗi kênh

bằng attach(pin, 1000, 2000), thư viện tạo xung PWM chuẩn cho ESC trong khoảng 1000–2000 µs.

1.17. Bộ điều khiển PID

PID (Proportional – Integral – Derivative) là một trong những phương pháp điều khiển phản hồi được sử dụng phổ biến nhất trong các hệ thống điều khiển tự động, bao gồm cả hệ thống cân bằng drone. Nguyên lý cơ bản của bộ điều khiển PID là tính toán sai số (error) giữa giá trị mong muốn (setpoint) và giá trị thực tế đo được (ví dụ góc nghiêng mong muốn và góc nghiêng thực tế đo từ MPU6050), sau đó dựa vào sai số này để tính ra tín hiệu điều khiển phù hợp.

Thành phần P (Proportional – tỉ lệ) tạo ra tín hiệu điều khiển tỉ lệ thuận với độ lớn của sai số hiện tại. Sai số càng lớn, tín hiệu điều chỉnh càng mạnh. Thành phần P giúp hệ thống phản ứng nhanh với sai số, tuy nhiên nếu chỉ sử dụng riêng thành phần P, hệ thống thường không thể triệt tiêu hoàn toàn sai số ở trạng thái ổn định (steady-state error).

Thành phần I (Integral – tích phân) tính tổng tích lũy của sai số theo thời gian, giúp triệt tiêu sai số ở trạng thái ổn định mà thành phần P không thể xử lý được. Tuy nhiên, nếu hệ số I quá lớn, thành phần tích phân có thể gây ra hiện tượng dao động hoặc vọt lố (overshoot) do sai số tích lũy quá nhiều trước khi hệ thống kịp phản ứng.

Thành phần D (Derivative – vi phân) tính tốc độ thay đổi của sai số theo thời gian, có tác dụng dự đoán xu hướng thay đổi trong tương lai gần và giúp giảm hiện tượng dao động, vọt lố do hai thành phần P và I gây ra. Tuy nhiên, thành phần D khá nhạy cảm với nhiễu trong tín hiệu đo, do đó nếu dữ liệu cảm biến bị nhiễu nhiều, thành phần D có thể khiến tín hiệu điều khiển dao động bất thường.

Trong hàm controlANGLE(), roll và pitch dùng sai số góc giữa giá trị đặt và góc Madgwick, còn yaw dùng sai số tốc độ giữa yaw_des và GyroZ. Thành phần tích phân được đưa về 0 khi throttle dưới 1060 µs và bị giới hạn trong khoảng ±25 để tránh tích lũy quá lớn.

Bộ thông số đang được khai báo trong mã nguồn gồm: roll góc Kp = 0,2, Ki = 0,3, Kd = 0,05; pitch góc Kp = 0,2, Ki = 0,3, Kd = 0,05; yaw tốc độ Kp = 0,3, Ki = 0,05, Kd = 0,00015. Các hệ số rate cho roll và pitch cũng được khai báo nhưng không được sử dụng vì vòng lặp đang gọi controlANGLE().

CHƯƠNG II. PHÂN TÍCH, THIẾT KẾ HỆ THỐNG, PHẦN CỨNG VÀ PHẦN MỀM

Chương này trình bày các yêu cầu của hệ thống, sơ đồ khối, nguyên lý hoạt động, hệ thống điều khiển bằng tay cầm, cấu hình phần cứng và cấu trúc phần mềm.

2.1. Yêu cầu chức năng

Hệ thống drone cần đáp ứng các yêu cầu chức năng sau:

- Đọc dữ liệu từ cảm biến MPU6050 theo thời gian thực.
- Lọc dữ liệu MPU6050 và ước lượng tư thế bằng Madgwick 6DOF; roll và pitch được dùng làm phản hồi góc, còn yaw được ước lượng từ gyro và có thể bị trôi.
- Nhận sáu kênh xung PWM từ tay cầm Tx thông qua mạch thu Rx bằng các ngắt GPIO.
- Đọc và xử lý các lệnh throttle, roll, pitch và yaw từ các kênh tín hiệu của mạch thu Rx.
- Điều khiển bốn ESC bằng thư viện ESP32Servo với bốn chân GPIO riêng và khoảng xung cấu hình 1000–2000 µs.
- Thực hiện PID góc cho roll và pitch, PID tốc độ cho yaw, sau đó trộn tín hiệu để điều khiển bốn động cơ.

Sử dụng CH5 làm kênh throttle cut; động cơ chỉ được cho phép khi CH5 ở mức thấp và throttle gần mức nhỏ nhất.
Kiểm tra giá trị sáu kênh thu; nếu một kênh nằm ngoài 800–2200 µs thì chương trình thay toàn bộ lệnh bằng bộ giá trị failsafe đã khai báo.

2.2. Yêu cầu phi chức năng

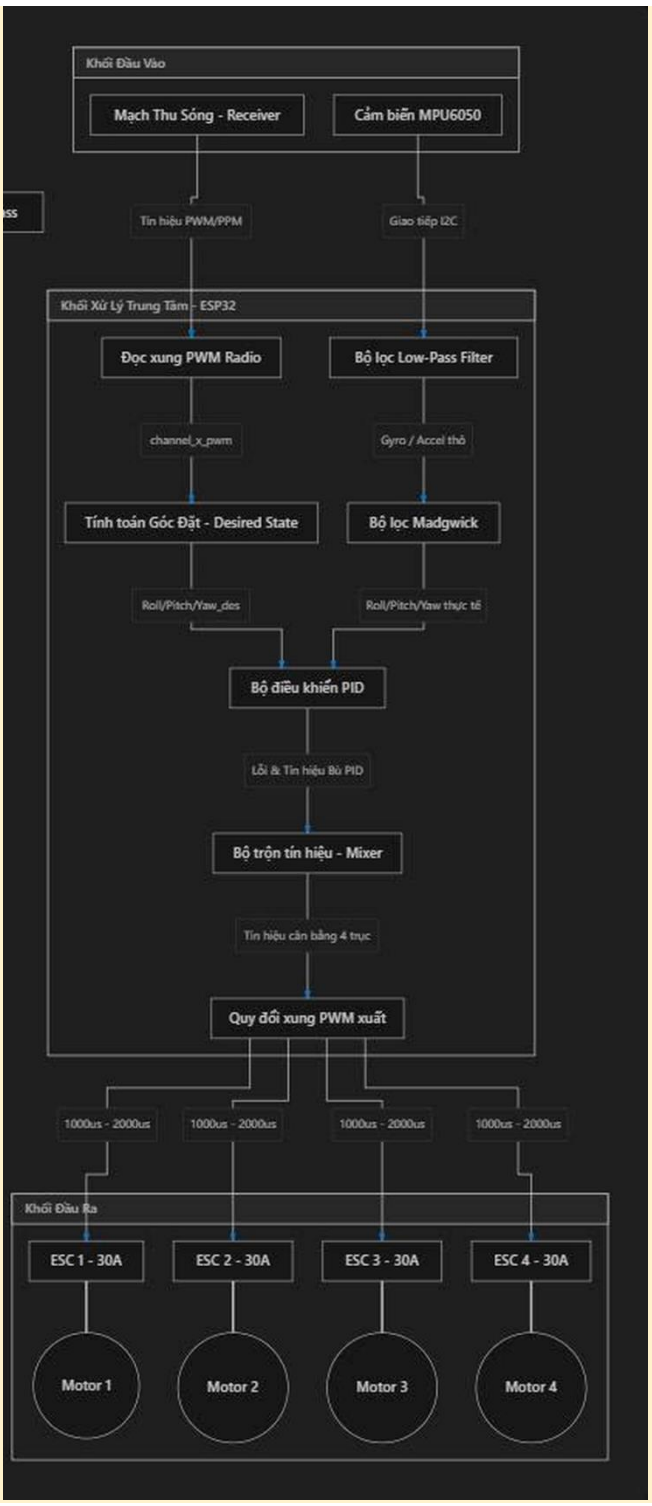
Bên cạnh các yêu cầu chức năng, hệ thống cũng cần đáp ứng một số yêu cầu phi chức năng nhằm bảo đảm tính khả thi trong điều kiện thực hiện đồ án sinh viên:

- Linh kiện dễ tìm mua, ưu tiên các nguồn cung cấp phổ biến như Shopee.
- Hệ thống dễ sửa chữa, thay thế linh kiện khi xảy ra hư hỏng.
- Tay cầm điều khiển dễ thao tác và các kênh tín hiệu phản hồi đúng theo chuyển động của cần điều khiển.
- Hạn chế khối lượng tổng thể của drone để phù hợp với lực nâng của động cơ được chọn.
- Hoàn thành trong thời gian còn lại của nhóm, khoảng 2–3 tuần.

2.3. Sơ đồ khối hệ thống

Hệ thống gồm tay cầm Microzone 6C Mini, mạch thu Microzone MC7RE-V2, ESP32, MPU6050, bộ lọc thông thấp, Madgwick 6DOF, PID, motor mixer, bốn ESC 30A, bốn động cơ A2212 2450KV, pin LiPo SHANG YI 3S 11,1 V – 2200 mAh và LM2596.

Luồng điều khiển gồm sáu xung PWM từ Rx đi vào ESP32, được lọc và chuẩn hóa thành throttle, roll_des, pitch_des, yaw_des. Luồng phản hồi gồm dữ liệu MPU6050 được lọc và xử lý bằng Madgwick 6DOF. PID và mixer tạo bốn lệnh chuẩn hóa, sau đó ESP32Servo phát xung 1000–2000 µs đến các ESC.



Hình 10: Sơ đồ khối xử lý và điều khiển của hệ thống drone

2.4. Nguyên lý hoạt động tổng thể

Nguyên lý hoạt động của hệ thống được mô tả theo trình tự các bước sau, lặp lại liên tục trong suốt quá trình vận hành:

– Bước 1: Pin LiPo SHANG YI 3S 11,1 V – 2200 mAh cấp nguồn công suất cho bốn ESC; nhánh nguồn qua LM2596 cấp 5 V cho ESP32 và mạch thu Rx.

- Bước 2: ESP32 khởi tạo Serial ở 115200 baud, gắn bốn đối tượng ESP32Servo vào GPIO26, GPIO33, GPIO14 và GPIO27 với khoảng xung 1000–2000 µs, đồng thời cấu hình sáu chân Rx ở chế độ INPUT.
- Bước 3: Tay cầm Microzone 6C Mini phát lệnh điều khiển ở băng tần 2,4 GHz; mạch thu Microzone MC7RE-V2 nhận và xuất các kênh tín hiệu đến ESP32.
- Bước 4: Các hàm ngắt đo thời điểm cạnh lên và cạnh xuống của từng kênh để tính độ rộng xung CH1–CH6; getCommands() đọc và lọc bốn kênh chính với hệ số b = 0,7.
- Bước 5: getDesState() đổi CH1 thành throttle 0–1, CH2 thành roll ±30°, CH3 thành pitch ±30° và CH4 thành tốc độ yaw ±160°/s.
- Bước 6: ESP32 quét bus I²C, khởi tạo MPU6050 tại địa chỉ 0x68, cấu hình gyro ±250°/s và accelerometer ±2 g, sau đó đọc trực tiếp 14 byte từ thanh ghi 0x3B.
- Bước 7: Dữ liệu được trừ offset, lọc thông thấp rồi đưa vào Madgwick6DOF để cập nhật quaternion và các góc roll_IMU, pitch_IMU, yaw_IMU.
- Bước 8: controlANGLE() tính PID góc cho roll và pitch, đồng thời tính PID tốc độ cho yaw dựa trên GyroZ.
- Bước 9: Bộ trộn tín hiệu (mixer) kết hợp throttle với các thành phần PID để tính giá trị điều khiển cân bằng cho bốn động cơ.
- Bước 10: scaleCommands() đổi bốn giá trị mixer sang miền 0–180; ESP32Servo ánh xạ miền này vào khoảng xung 1000–2000 µs đã khai báo khi attach().
- Bước 11: throttleCut() kiểm tra CH5 và trạng thái armedFly trước khi các lệnh được ghi ra ESC; failSafe() thay lệnh bằng giá trị mặc định nếu xung Rx vượt ngoài 800–2200 µs.

2.5. Thiết kế hệ thống điều khiển bằng tay cầm

Hệ thống sử dụng tay cầm Microzone 6C Mini 2,4 GHz, 6 kênh và mạch thu Microzone MC7RE-V2. Mã nguồn chọn chế độ USE_PWM_RX, tức là mỗi kênh Rx được đọc bằng một GPIO riêng.

- CH1 là throttle và được nối vào GPIO35; giá trị khoảng 1000–2000 µs được chuẩn hóa về 0–1.
- CH2 (AIL) nối GPIO18 để tạo roll_des; CH3 (ELE) nối GPIO17 để tạo pitch_des.
- CH4 (RUD) nối GPIO19 để tạo yaw_des; CH5 nối GPIO16 làm throttle cut; CH6 nối GPIO32 là kênh phụ.

– Mỗi kênh được đo bằng ngắt CHANGE. Khi tín hiệu lên mức HIGH, chương trình lưu micros(); khi xuống LOW, chương trình lấy hiệu thời gian để thu được độ rộng xung.

– ESP32 nhận sáu kênh tín hiệu Rx qua GPIO35, GPIO18, GPIO17, GPIO19, GPIO16 và GPIO32.

– Bảng ánh xạ trong báo cáo được lấy theo các hằng ch1Pin–ch6Pin của mã nguồn; khi lắp thực tế cần cắm đúng thứ tự CH1–CH6 tương ứng.

– Trước khi vận hành, nhóm kiểm tra chiều tăng giảm và khoảng độ rộng xung của từng kênh trên Serial Monitor để bảo đảm dữ liệu được đọc đúng.



Hình 11: Tay cầm điều khiển Microzone 6C

2.6. Lựa chọn cấu hình phần cứng

Nhóm lựa chọn cấu hình gồm bốn động cơ brushless A2212 2450KV, bốn ESC 30A, cánh quạt 5 inch 5045 và pin LiPo SHANG YI 3S 11,1 V – 2200 mAh lắp trên khung F250.

2.7. Danh sách linh kiện

Bảng 4.1 trình bày danh sách linh kiện thực tế được nhóm sử dụng trong hệ thống, cùng mã hoặc thông số, số lượng và công dụng của từng linh kiện.

STT	Tên linh kiện	Mã / thông số	SL	Công dụng
1	Khung sườn F250	F250	1 bộ	Khung cơ khí lắp các linh kiện
2	Động cơ Brushless A2212 2450KV	A2212 2450KV	4 chiếc	Tạo lực đẩy
3	Mạch điều tốc ESC 30A	30A	4 chiếc	Điều khiển tốc độ động cơ brushless
4	Cánh quạt 5 inch 5045	5045	4 chiếc	Tạo lực nâng
5	Pin LiPo SHANG YI	3S, 11,1 V, 2200 mAh, đầu XT60	1 cục	Cấp nguồn cho bốn ESC và hệ thống qua LM2596
6	Dây giắc cắm nguồn XT60 đực – cái	XT60	3 bộ	Kết nối và phân phối nguồn
7	Mạch giảm áp DC-DC LM2596	LM2596	1 chiếc	Hạ áp cấp cho ESP32 và mạch thu Rx
8	Mạch vi điều khiển ESP32 DevKit V1	ESP32 DevKit V1	1 chiếc	Vi điều khiển trung tâm
9	Cảm biến MPU6050	MPU6050	1 chiếc	Đo gia tốc và vận tốc góc
10	Tay cầm điều khiển Microzone 6C Mini	2,4 GHz, 6 kênh	1 chiếc	Phát lệnh điều khiển
11	Mạch thu sóng Microzone MC7RE-V2	Receiver – Rx	1 chiếc	Nhận tín hiệu và xuất các kênh PWM đến ESP32
12	Breadboard và dây cắm Dupont	Breadboard, dây Dupont	1 bộ	Kết nối mạch

2.8. Sơ đồ kết nối ESP32 và MPU6050

MPU6050 giao tiếp với ESP32 thông qua chuẩn I²C, sử dụng bốn đường tín hiệu chính: VCC (nguồn cấp, thường là 3.3V), GND (nối đất chung), SDA (đường truyền dữ liệu) và SCL (đường xung nhịp). Việc xác định đúng các chân GPIO trên ESP32 được sử dụng cho SDA và SCL là yêu cầu quan trọng để bảo đảm giao tiếp I²C hoạt động chính xác.

2.9. Sơ đồ mạch điều khiển động cơ

Hệ thống sử dụng bốn động cơ brushless A2212 2450KV và bốn mạch điều tốc ESC 30A.

Theo các hằng servo1Pin–servo4Pin, ESC 1, ESC 2, ESC 3 và ESC 4 lần lượt nhận tín hiệu từ GPIO26, GPIO33, GPIO14 và GPIO27.

Nguồn của bốn ESC được nối với pin LiPo SHANG YI 3S 11,1 V – 2200 mAh qua hệ thống dây và giắc XT60, còn ba dây pha của từng ESC được nối với động cơ A2212 2450KV tương ứng.

2.10. Hệ thống nguồn

Hệ thống nguồn sử dụng pin LiPo SHANG YI 3S 11,1 V – 2200 mAh cấp trực tiếp cho bốn ESC. Một nhánh nguồn được đưa qua mạch giảm áp DC-DC LM2596 để tạo 5 V cấp cho ESP32 và mạch thu Microzone MC7RE-V2; MPU6050 nhận nguồn 3,3 V từ ESP32.

Mặc dù hai đường cấp nguồn được tách riêng để giảm nhiễu, toàn bộ hệ thống vẫn cần có một điểm nối đất (GND) chung giữa các khối, nhằm bảo đảm tín hiệu điều khiển (như PWM) từ ESP32 có cùng điểm tham chiếu điện áp với mạch công suất động cơ.

2.11. Khung và bố trí linh kiện

Khung được nhóm sử dụng là khung sườn F250, có nhiệm vụ cố định bốn động cơ, bốn ESC, pin, ESP32, MPU6050, mạch thu Rx và mạch giảm áp.

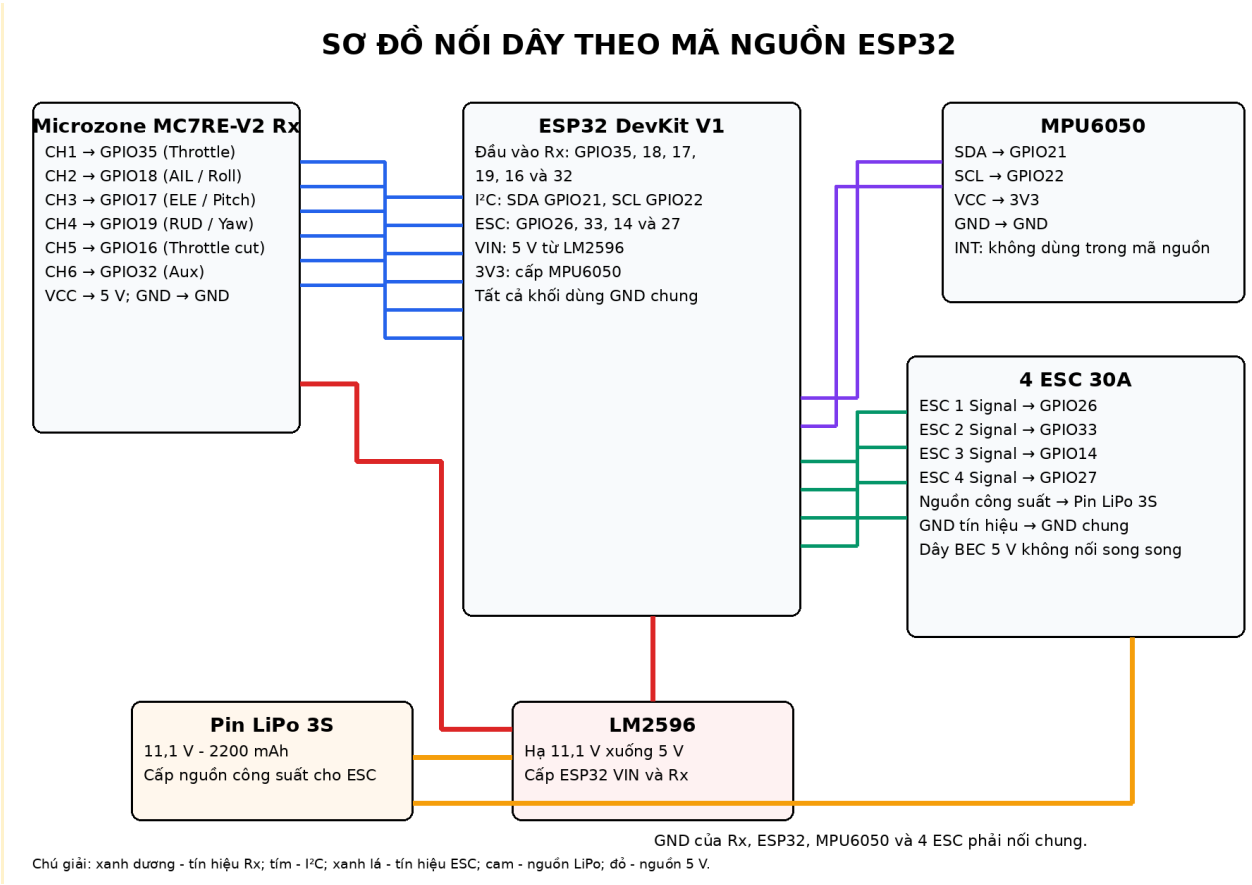
Các linh kiện cần được bố trí sao cho trọng tâm nằm gần tâm khung; pin nên được cố định chắc chắn, MPU6050 đặt gần tâm và hạn chế rung, còn các ESC được bố trí gần động cơ tương ứng để giảm chiều dài dây công suất.



Hình 12: Bố trí các linh kiện chính trên khung drone

2.12. Sơ đồ đấu nối toàn hệ thống

Hình 4.6 được cập nhật theo khai báo chân trong mã nguồn. MPU6050 dùng SDA GPIO21 và SCL GPIO22; Rx dùng GPIO35, 18, 17, 19, 16, 32; bốn ESC dùng GPIO26, 33, 14, 27. Bốn ESC lấy nguồn công suất từ pin LiPo; dây BEC 5 V không nối song song vào ESP32.



Hình 13: Sơ đồ nối dây được cập nhật theo mã nguồn của nhóm

2.13. Bảng chân kết nối

Bảng 4.2 tổng hợp các chân GPIO của ESP32 được sử dụng trong hệ thống và chức năng tương ứng.

Chức năng	Linh kiện	Chân ESP32	Ghi chú
I²C – SDA	MPU6050	GPIO21	Wire.begin(21, 22)
I²C – SCL	MPU6050	GPIO22	I²C 400 kHz
Tín hiệu ESC 1	ESC 1	GPIO26	servo1Pin; Front Left (s1)
Tín hiệu ESC 2	ESC 2	GPIO33	servo2Pin; Front Right (s2)
Tín hiệu ESC 3	ESC 3	GPIO14	servo3Pin; Back Right (s3)
Tín hiệu ESC 4	ESC 4	GPIO27	servo4Pin; Back Left (s4)
Rx CH1	Microzone MC7RE-V2	GPIO35	Throttle
Rx CH2	Microzone MC7RE-V2	GPIO18	AIL / Roll
Rx CH3	Microzone MC7RE-V2	GPIO17	ELE / Pitch

Chức năng	Linh kiện	Chân ESP32	Ghi chú
Rx CH4	Microzone MC7RE-V2	GPIO19	RUD / Yaw
Rx CH5	Microzone MC7RE-V2	GPIO16	Gear / Throttle cut
Rx CH6	Microzone MC7RE-V2	GPIO32	Aux
LED trạng thái	ESP32 DevKit V1	GPIO13	Báo trạng thái chương trình
Nguồn cảm biến	MPU6050	3.3V	INT không dùng trong code
Nguồn vào ESP32	LM2596	VIN 5V	Nguồn 5 V ổn định
Nguồn mạch thu	Microzone MC7RE-V2	5V	Nguồn từ LM2596
Nối đất chung	ESC, MPU6050, Rx, LM2596	GND	Các khối phải chung GND

Phần này trình bày cấu trúc chương trình chạy trên ESP32, bao gồm các module xử lý MPU6050, bộ lọc thông thấp, thuật toán Madgwick, đọc tín hiệu từ mạch thu Rx, tính góc đặt, PID, motor mixer và điều khiển bốn ESC.

2.14. Môi trường phát triển

Mã nguồn được viết bằng C/C++ theo Arduino framework và có phần khai báo prototype dành cho PlatformIO. Các thư viện chính gồm Arduino.h, Wire.h, SPI.h, ESP32Servo.h và tệp MPU6050/MPU6050.h; Serial Monitor chạy ở 115200 baud.

2.15. Cấu trúc chương trình

Chương trình điều khiển được tổ chức theo hướng chia thành các module chức năng riêng biệt nhằm thuận tiện cho việc phát triển, kiểm thử và bảo trì. Các module chính bao gồm:

- Module khởi tạo hệ thống: cấu hình các ngoại vi ban đầu khi ESP32 khởi động.
- Module giao tiếp MPU6050: khởi tạo và đọc dữ liệu cảm biến qua I²C.
- Module Rx PWM: cấu hình sáu GPIO INPUT, gắn ngắt CHANGE và đo độ rộng xung bằng micros().
- Module getCommands()/failSafe(): đọc sáu kênh, lọc bốn kênh chính và thay giá trị mặc định nếu xung ngoài 800–2200 µs.
- Module getDesState(): chuẩn hóa throttle, roll, pitch, yaw theo các giới hạn 0–1, ±30° và ±160°/s.

– Module ESC: dùng ESP32Servo trên GPIO26, GPIO33, GPIO14, GPIO27; giá trị 0–180 được ánh xạ sang xung 1000–2000 µs.
– Module IMU: đọc trực tiếp 14 byte từ MPU6050, trừ offset, lọc thông thấp và chạy Madgwick6DOF.
– Module controlANGLE()/controlMixer(): PID roll, pitch, yaw và trộn tín hiệu cho bốn động cơ.
– Module trạng thái đầu ra: armedStatus(), throttleCut(), armMotors() và calibrateESCs().

2.16. Khởi tạo MPU6050

Trong setup(), chương trình gọi Wire.begin(21, 22), đặt timeout 100 ms và quét toàn bộ địa chỉ I²C. Nếu không phát hiện thiết bị, chương trình dừng tại while(1). IMUinit() đánh thức MPU6050 ở địa chỉ 0x68, đặt clock I²C 400 kHz, cấu hình gyro ±250°/s và accelerometer ±2 g.

Đoạn mã 2.1. Khởi tạo, quét I²C và cấu hình MPU6050

```
// Cấu hình phần cứng MPU6050
#define USE_MPU6050_I2C
#define GYRO_250DPS
#define ACCEL_2G

Wire.begin(21, 22);      // SDA = GPIO21, SCL = GPIO22
Wire.setTimeOut(100);

// Quét bus I2C
for (byte address = 1; address < 127; address++) {
  Wire.beginTransmission(address);
  if (Wire.endTransmission() == 0) {
    Serial.print("Da tim thay thiet bi I2C: 0x");
    Serial.println(address, HEX);
  }
}

void IMUinit() {
  Wire.begin(21, 22);
  Wire.setClock(400000);

  Wire.beginTransmission(0x68);
  Wire.write(0x6B);       // PWR_MGMT_1
  Wire.write(0x01);       // Đánh thức MPU6050
  Wire.endTransmission();

  Wire.beginTransmission(0x68);
  Wire.write(0x1B);       // GYRO_CONFIG
  Wire.write(GYRO_SCALE); // ±250 dps
  Wire.endTransmission();

  Wire.beginTransmission(0x68);
  Wire.write(0x1C);       // ACCEL_CONFIG
  Wire.write(ACCEL_SCALE);// ±2 g
  Wire.endTransmission();
}
```

2.17. Hiệu chuẩn cảm biến

Hàm calculate_IMU_error() lấy 12000 mẫu, tính trung bình và in các offset để chép vào vùng biến cấu hình. Trong phiên bản được cung cấp, lời gọi hàm này đang bị

comment và chương trình dùng trực tiếp các giá trị AccErrorX = 0,02; AccErrorY = -0,12; AccErrorZ = -2,00; GyroErrorX = -1,02; GyroErrorY = 0,63; GyroErrorZ = 0,99.

Trong quá trình hiệu chuẩn, drone tuyệt đối không được di chuyển hoặc rung lắc, vì bất kỳ chuyển động nào trong giai đoạn này cũng sẽ được ghi nhận nhầm thành độ lệch của cảm biến, dẫn đến sai số trong toàn bộ quá trình tính toán góc sau này.

Đoạn mã 2.2. Giá trị offset và quy trình lấy 12000 mẫu hiệu chuẩn

```
// Offset đang được sử dụng trong chương trình
float AccErrorX = 0.02;
float AccErrorY = -0.12;
float AccErrorZ = -2.00;
float GyroErrorX = -1.02;
float GyroErrorY = 0.63;
float GyroErrorZ = 0.99;

// Trong calculate_IMU_error():
int c = 0;
while (c < 12000) {
  // Đọc AccX, AccY, AccZ, GyroX, GyroY, GyroZ
  AccErrorX  += AccX;
  AccErrorY  += AccY;
  AccErrorZ  += AccZ;
  GyroErrorX += GyroX;
  GyroErrorY += GyroY;
  GyroErrorZ += GyroZ;
  c++;
}

AccErrorX  = AccErrorX / c;
AccErrorY  = AccErrorY / c;
AccErrorZ  = AccErrorZ / c - 1.0;
GyroErrorX = GyroErrorX / c;
GyroErrorY = GyroErrorY / c;
GyroErrorZ = GyroErrorZ / c;
```

2.18. Lọc dữ liệu và ước lượng tư thế bằng Madgwick

getIMUdata() đọc 14 byte từ thanh ghi ACCEL_XOUT_H (0x3B), bỏ qua hai byte nhiệt độ, chia theo hệ số 16384 LSB/g và 131 LSB/(°/s), trừ offset rồi lọc bậc một với B_accel = 0,14 và B_gyro = 0,10.

Vì USE_MPU6050_I2C được bật, hàm Madgwick() chuyển sang Madgwick6DOF(). Quaternion q0–q3 được tích phân và chuẩn hóa, sau đó chuyển thành roll_IMU, pitch_IMU và yaw_IMU. Roll/pitch là phản hồi góc; yaw không có tham chiếu từ trường nên có thể trôi.

Đoạn mã 2.3. Lọc thông thấp và Madgwick 6DOF

```
float B_madgwick = 0.04;
float B_accel = 0.14;
float B_gyro = 0.10;

// Lọc thông thấp dữ liệu accelerometer
AccX = (1.0 - B_accel) * AccX_prev + B_accel * AccX;
AccY = (1.0 - B_accel) * AccY_prev + B_accel * AccY;
AccZ = (1.0 - B_accel) * AccZ_prev + B_accel * AccZ;

// Lọc thông thấp dữ liệu gyroscope
GyroX = (1.0 - B_gyro) * GyroX_prev + B_gyro * GyroX;
GyroY = (1.0 - B_gyro) * GyroY_prev + B_gyro * GyroY;
GyroZ = (1.0 - B_gyro) * GyroZ_prev + B_gyro * GyroZ;

// MPU6050 không có từ kế nên dùng Madgwick 6DOF
```

```
Madgwick6DOF(gx, gy, gz, ax, ay, az, dt);

// Chuyển quaternion thành góc
roll_IMU  = atan2(q0*q1 + q2*q3,
                  0.5f - q1*q1 - q2*q2) * 57.29577951;
pitch_IMU = -asin(constrain(-2.0f * (q1*q3 - q0*q2),
                            -0.999999, 0.999999)) * 57.29577951;
yaw_IMU   = -atan2(q1*q2 + q0*q3,
                   0.5f - q2*q2 - q3*q3) * 57.29577951;
```

2.19. Khởi tạo bộ thu tín hiệu (Rx)

radioSetup() cấu hình sáu chân nhận xung: GPIO35, GPIO18, GPIO17, GPIO19, GPIO16 và GPIO32; mỗi chân được gắn một ISR ở chế độ CHANGE.

Đoạn mã 2.4. Khai báo chân và ngắt đọc sáu kênh Rx

```
// Khai báo 6 kênh PWM từ mạch thu
const int ch1Pin = 35; // Throttle
const int ch2Pin = 18; // AIL / Roll
const int ch3Pin = 17; // ELE / Pitch
const int ch4Pin = 19; // RUD / Yaw
const int ch5Pin = 16; // Gear / Throttle cut
const int ch6Pin = 32; // Aux

void radioSetup() {
  pinMode(ch1Pin, INPUT);
  pinMode(ch2Pin, INPUT);
  pinMode(ch3Pin, INPUT);
  pinMode(ch4Pin, INPUT);
  pinMode(ch5Pin, INPUT);
  pinMode(ch6Pin, INPUT);

  attachInterrupt(digitalPinToInterrupt(ch1Pin), ch1_isr, CHANGE);
  attachInterrupt(digitalPinToInterrupt(ch2Pin), ch2_isr, CHANGE);
  attachInterrupt(digitalPinToInterrupt(ch3Pin), ch3_isr, CHANGE);
  attachInterrupt(digitalPinToInterrupt(ch4Pin), ch4_isr, CHANGE);
  attachInterrupt(digitalPinToInterrupt(ch5Pin), ch5_isr, CHANGE);
  attachInterrupt(digitalPinToInterrupt(ch6Pin), ch6_isr, CHANGE);
}

// Ví dụ ISR đo độ rộng xung CH1
void IRAM_ATTR ch1_isr() {
  if (digitalRead(ch1Pin) == HIGH) ch1_start = micros();
  else ch1_val = micros() - ch1_start;
}
```

2.20. Đọc và xử lý tín hiệu tay cầm

Các ISR lưu thời điểm cạnh lên và tính độ rộng xung ở cạnh xuống. getRadioPWM() trả về giá trị từng kênh; getCommands() cập nhật sáu biến channel_x_pwm.

Bốn kênh CH1–CH4 được lọc bậc một với b = 0,7. failSafe() kiểm tra giới hạn 800–2200 µs; nếu bất kỳ kênh nào không hợp lệ, toàn bộ sáu kênh được đưa về giá trị mặc định.

Đoạn mã 2.5. Đọc, lọc và kiểm tra sáu kênh PWM

```
void getCommands() {
  channel_1_pwm = getRadioPWM(1);
  channel_2_pwm = getRadioPWM(2);
  channel_3_pwm = getRadioPWM(3);
  channel_4_pwm = getRadioPWM(4);
  channel_5_pwm = getRadioPWM(5);
  channel_6_pwm = getRadioPWM(6);

  // Lọc thông thấp bốn kênh chính
  float b = 0.7;
  channel_1_pwm = (1.0 - b)*channel_1_pwm_prev + b*channel_1_pwm;
  channel_2_pwm = (1.0 - b)*channel_2_pwm_prev + b*channel_2_pwm;
  channel_3_pwm = (1.0 - b)*channel_3_pwm_prev + b*channel_3_pwm;
  channel_4_pwm = (1.0 - b)*channel_4_pwm_prev + b*channel_4_pwm;
```

```
}

void failSafe() {
  unsigned minVal = 800;
  unsigned maxVal = 2200;

  if (channel_1_pwm < minVal || channel_1_pwm > maxVal ||
      channel_2_pwm < minVal || channel_2_pwm > maxVal ||
      channel_3_pwm < minVal || channel_3_pwm > maxVal ||
      channel_4_pwm < minVal || channel_4_pwm > maxVal ||
      channel_5_pwm < minVal || channel_5_pwm > maxVal ||
      channel_6_pwm < minVal || channel_6_pwm > maxVal) {
    channel_1_pwm = 1000;
    channel_2_pwm = 1500;
    channel_3_pwm = 1500;
    channel_4_pwm = 1500;
    channel_5_pwm = 2000;
    channel_6_pwm = 2000;
  }
}
```

2.21. Chuyển đổi tín hiệu điều khiển

getDesState() thực hiện: thro_des = (CH1−1000)/1000; roll_des = (CH2−1500)/500×30°; pitch_des = (CH3−1500)/500×30°; yaw_des = (CH4−1500)/500×160°/s, sau đó giới hạn từng đại lượng.

Đoạn mã 2.6. Chuẩn hóa lệnh throttle, roll, pitch và yaw

```
void getDesState() {
  //DESCRIPTION: Normalizes desired control values to appropriate values
  /*
   * Updates the desired state variables thro_des, roll_des, pitch_des, and yaw_des. These are computed by using
the raw
   * RC pwm commands and scaling them to be within our limits defined in setup. thro_des stays within 0 to 1
range.
   * roll_des and pitch_des are scaled to be within max roll/pitch amount in either degrees (angle mode) or
degrees/sec
   * (rate mode). yaw_des is scaled to be within max yaw in degrees/sec. Also creates roll_passthru,
pitch_passthru, and
   * yaw_passthru variables, to be used in commanding motors/servos with direct unstabilized commands in
controlMixer().
   */
  thro_des = (channel_1_pwm - 1000.0)/1000.0; //Between 0 and 1
  roll_des = (channel_2_pwm - 1500.0)/500.0; //Between -1 and 1
  pitch_des = (channel_3_pwm - 1500.0)/500.0; //Between -1 and 1
  yaw_des = (channel_4_pwm - 1500.0)/500.0; //Between -1 and 1
  roll_passthru = roll_des/2.0; //Between -0.5 and 0.5
  pitch_passthru = pitch_des/2.0; //Between -0.5 and 0.5
  yaw_passthru = yaw_des/2.0; //Between -0.5 and 0.5

  //Constrain within normalized bounds
  thro_des = constrain(thro_des, 0.0, 1.0); //Between 0 and 1
  roll_des = constrain(roll_des, -1.0, 1.0)*maxRoll; //Between -maxRoll and +maxRoll
  pitch_des = constrain(pitch_des, -1.0, 1.0)*maxPitch; //Between -maxPitch and +maxPitch
  yaw_des = constrain(yaw_des, -1.0, 1.0)*maxYaw; //Between -maxYaw and +maxYaw
  roll_passthru = constrain(roll_passthru, -0.5, 0.5);
  pitch_passthru = constrain(pitch_passthru, -0.5, 0.5);
  yaw_passthru = constrain(yaw_passthru, -0.5, 0.5);
}
```

2.22. Điều khiển tốc độ động cơ

Bốn đối tượng Servo được gắn vào GPIO26, GPIO33, GPIO14 và GPIO27 bằng attach(pin, 1000, 2000). Mixer tạo giá trị chuẩn hóa, scaleCommands() đổi sang 0–180 và loop() ghi bằng servo1.write() đến servo4.write().

Đoạn mã 2.7. Khởi tạo ESP32Servo và xuất lệnh đến bốn ESC

```
// Bốn đầu ra PWM thực sự dùng cho ESC 30A
const int servo1Pin = 26;
const int servo2Pin = 33;
const int servo3Pin = 14;
```

```
const int servo4Pin = 27;

Servo servo1;
Servo servo2;
Servo servo3;
Servo servo4;

void setup() {
  servo1.attach(servo1Pin, 1000, 2000);
  servo2.attach(servo2Pin, 1000, 2000);
  servo3.attach(servo3Pin, 1000, 2000);
  servo4.attach(servo4Pin, 1000, 2000);
}

void loop() {
  // s1_command_PWM ... s4_command_PWM nằm trong khoảng 0-180
  servo1.write(s1_command_PWM);
  servo2.write(s2_command_PWM);
  servo3.write(s3_command_PWM);
  servo4.write(s4_command_PWM);
}
```

2.23. Thuật toán PID

controlANGLE() dùng PID góc cho roll và pitch; đạo hàm lấy trực tiếp GyroX/GyroY. Yaw dùng PID tốc độ với sai số yaw_des−GyroZ. Tích phân bị xóa khi CH1 < 1060 µs và giới hạn bởi i_limit = 25.

Đoạn mã 2.8. PID góc roll/pitch và PID tốc độ yaw

```
// Thông số PID đang hoạt động
float i_limit = 25.0;
float maxRoll = 30.0;
float maxPitch = 30.0;
float maxYaw = 160.0;

float Kp_roll_angle = 0.2;
float Ki_roll_angle = 0.3;
float Kd_roll_angle = 0.05;
float Kp_pitch_angle = 0.2;
float Ki_pitch_angle = 0.3;
float Kd_pitch_angle = 0.05;
float Kp_yaw = 0.3;
float Ki_yaw = 0.05;
float Kd_yaw = 0.00015;

void controlANGLE() {
  error_roll = roll_des - roll_IMU;
  integral_roll += error_roll * dt;
  if (channel_1_pwm < 1060) integral_roll = 0;
  integral_roll = constrain(integral_roll, -i_limit, i_limit);
  derivative_roll = GyroX;
  roll_PID = 0.01 * (Kp_roll_angle*error_roll
           + Ki_roll_angle*integral_roll
           - Kd_roll_angle*derivative_roll);

  error_pitch = pitch_des - pitch_IMU;
  integral_pitch += error_pitch * dt;
  if (channel_1_pwm < 1060) integral_pitch = 0;
  integral_pitch = constrain(integral_pitch, -i_limit, i_limit);
  derivative_pitch = GyroY;
  pitch_PID = 0.01 * (Kp_pitch_angle*error_pitch
            + Ki_pitch_angle*integral_pitch
            - Kd_pitch_angle*derivative_pitch);

  error_yaw = yaw_des - GyroZ;
  integral_yaw += error_yaw * dt;
  if (channel_1_pwm < 1060) integral_yaw = 0;
  integral_yaw = constrain(integral_yaw, -i_limit, i_limit);
  derivative_yaw = (error_yaw - error_yaw_prev) / dt;
  yaw_PID = 0.01 * (Kp_yaw*error_yaw
          + Ki_yaw*integral_yaw
          + Kd_yaw*derivative_yaw);
}
```

Thông số đang hoạt động: Roll (0,2; 0,3; 0,05), Pitch (0,2; 0,3; 0,05), Yaw (0,3; 0,05; 0,00015).

2.24. Motor mixer

controlMixer() sử dụng cấu hình chữ X: động cơ trước trái nhận throttle−pitch+roll+yaw; trước phải nhận throttle−pitch−roll−yaw; sau phải nhận throttle+pitch−roll+yaw; sau trái nhận throttle+pitch+roll−yaw.

Đoạn mã 2.9. Bộ trộn tín hiệu cho cấu hình quadcopter chữ X

```
void controlMixer() {
  //DESCRIPTION: Mixes scaled commands from PID controller to actuator outputs based on vehicle configuration
  /*
   * Takes roll_PID, pitch_PID, and yaw_PID computed from the PID controller and appropriately mixes them for
the desired
   * vehicle configuration...
   */

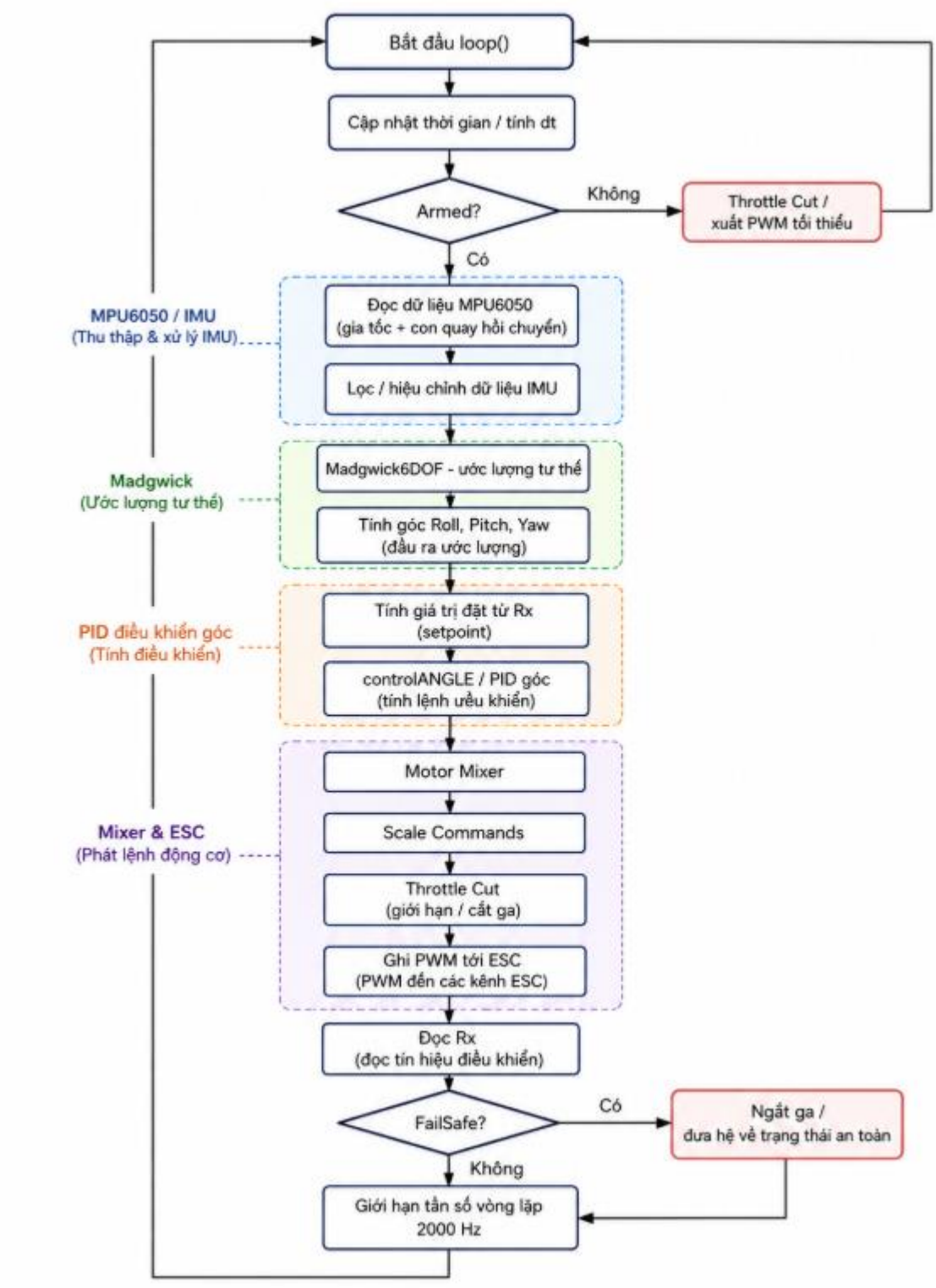
  //Quad mixing - ĐÃ CHUYỂN SANG DÙNG BIẾN 's' ĐỂ CHẠY CHUẨN PWM 50Hz CHO ESC 30A
  s1_command_scaled = thro_des - pitch_PID + roll_PID + yaw_PID; //Front Left
  s2_command_scaled = thro_des - pitch_PID - roll_PID - yaw_PID; //Front Right
  s3_command_scaled = thro_des + pitch_PID - roll_PID + yaw_PID; //Back Right
  s4_command_scaled = thro_des + pitch_PID + roll_PID - yaw_PID; //Back Left

  // Các chân servo còn lại không dùng đến
  s5_command_scaled = 0;
  s6_command_scaled = 0;
  s7_command_scaled = 0;

  // Triệt tiêu hoàn toàn luồng tín hiệu OneShot125 (m) để tránh gửi tín hiệu rác
  m1_command_scaled = 0;
  m2_command_scaled = 0;
  m3_command_scaled = 0;
  m4_command_scaled = 0;
  m5_command_scaled = 0;
  m6_command_scaled = 0;
}
```

2.25. Lưu đồ chương trình

Lưu đồ tương ứng với loop(): cập nhật thời gian → kiểm tra trạng thái armed → đọc/lọc IMU → Madgwick6DOF → tính giá trị đặt → controlANGLE → mixer → scaleCommands → throttleCut → ghi PWM ESC → đọc Rx → failSafe → giới hạn vòng lặp 2000 Hz.



Hình 14: Lưu đồ xử lý dữ liệu cảm biến và điều khiển động cơ

Đoạn mã 2.10. Trình tự xử lý chính trong loop()

```
void loop() {
  prev_time = current_time;
  current_time = micros();
  dt = (current_time - prev_time) / 1000000.0;

  armedStatus();
  getIMUdata();
  Madgwick(GyroX, -GyroY, -GyroZ,
           -AccX, AccY, AccZ,
           MagY, -MagX, MagZ, dt);
```

```
  getDesState();
  controlANGLE();
  controlMixer();
  scaleCommands();
  throttleCut();

  servo1.write(s1_command_PWM);
  servo2.write(s2_command_PWM);
  servo3.write(s3_command_PWM);
  servo4.write(s4_command_PWM);

  getCommands();
  failSafe();
  loopRate(2000);
}
```

CHƯƠNG III. QUÁ TRÌNH LẮP RÁP, THỰC HIỆN, KIỂM THỬ VÀ ĐÁNH GIÁ

Chương này mô tả trình tự thực hiện lắp ráp phần cứng và tích hợp hệ thống, từ giai đoạn chuẩn bị linh kiện, thử nghiệm từng thành phần riêng lẻ, cho đến khi hoàn thiện mô hình drone. Các nội dung trong chương này cần được nhóm bổ sung hình ảnh và

3.1. Thử từng động cơ

Trước khi lắp cánh quạt, nhóm hiệu chuẩn ESC bằng hàm calibrateESCs(), sau đó kiểm tra riêng từng ESC trên GPIO26, GPIO33, GPIO14 và GPIO27. Vì calibrateESCs() có vòng while(true), chỉ bật hàm khi hiệu chuẩn và phải tắt lời gọi này trước khi chạy vòng điều khiển bình thường.

```
315  void setup() {
394    delay(5);
395
396    m1_command_PWM = 125;
397    m2_command_PWM = 125;
398    m3_command_PWM = 125;
399    m4_command_PWM = 125;
400    m5_command_PWM = 125;
401    m6_command_PWM = 125;
402    armMotors();
403
404    setupBlink(3,160,70);
405
406    calibrateESCs();
407
408    Serial.println("KHOI DONG XONG! BAT DAU CHAY VONG LAP...");
409  }
410
411
412
413  //================================================================================
```

Hình 15: Code hiệu chuẩn ESC bằng hàm calibrateESCs

3.2. Lắp bốn động cơ

Sau khi từng động cơ đã được kiểm tra và xác nhận hoạt động đúng, cả bốn động cơ được lắp cố định lên khung drone theo đúng vị trí và chiều quay đã thiết kế.



Hình 16: Bốn động cơ đã được lắp đặt hoàn chỉnh trên khung

3.3. Lắp mạch và pin

Tiếp theo, ESP32, MPU6050, mạch thu MC7RE-V2, bốn ESC 30A, LM2596 và pin LiPo SHANG YI 3S 11,1 V – 2200 mAh được lắp lên khung F250, bảo đảm dây dẫn được cố định gọn gàng và không chạm vào cánh quạt.

3.4. Kiểm tra trọng tâm và chiều cánh

Trước khi tiến hành các bước thử nghiệm tiếp theo, nhóm cần kiểm tra một số yếu tố quan trọng: trọng tâm tổng thể của drone nên nằm gần với tâm hình học của khung để bảo đảm cân bằng cơ khí ban đầu; cánh quạt CW và CCW phải được lắp đúng vị trí tương ứng với chiều quay của từng động cơ theo thiết kế ở mục 1.8; dây dẫn cần được cố định chắc chắn, không để chạm vào cánh quạt khi quay; và cảm biến MPU6050 cần được cố định chắc chắn vào khung, đặt đúng hướng theo trục đã quy định trong mã nguồn, tránh trường hợp cảm biến bị lệch hướng dẫn đến sai số trong quá trình tính toán góc.

3.5. Những khó khăn trong quá trình thực hiện

Trong quá trình thực hiện đồ án, nhóm dự kiến sẽ gặp phải một số khó khăn về kỹ thuật cũng như thời gian thực hiện. Việc ghi nhận đầy đủ và trung thực các khó khăn thực tế, cùng với cách khắc phục, là một phần quan trọng của báo cáo nhằm phản ánh đúng quá trình học tập và thực hành của nhóm. Một số nhóm khó khăn thường gặp trong các đồ án tương tự bao gồm:

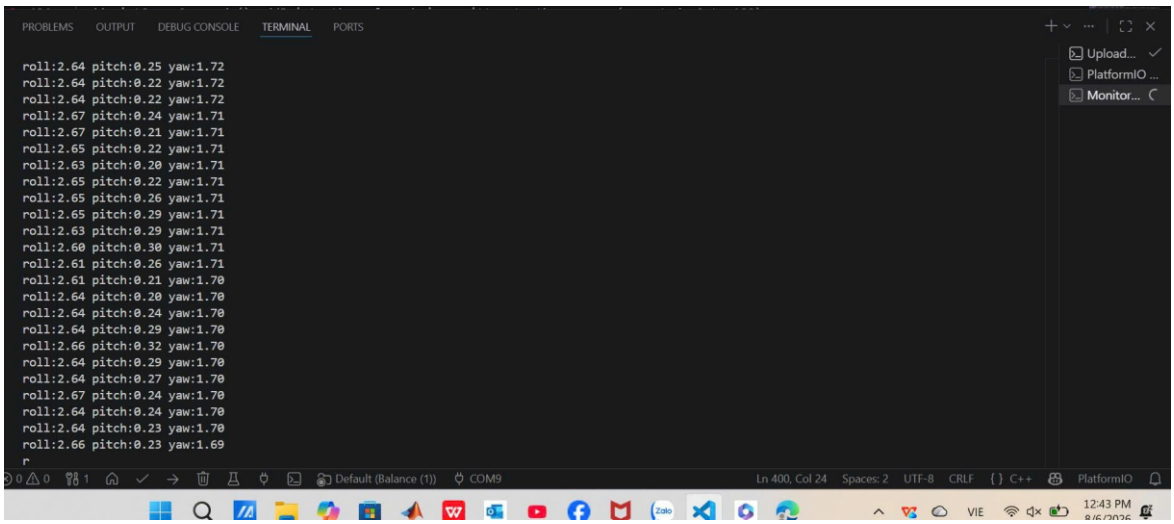
- Khó tìm khung phù hợp với kích thước và khối lượng của các linh kiện đã chọn.
- Motor và khung không tương thích về kích thước lỗ bắt vít hoặc đường kính trục.
- Sai lệch giữa lỗ trục cánh quạt và trục động cơ.
- Nhiễu dữ liệu cảm biến MPU6050 do rung động từ động cơ.
- ESP32 bị reset hoặc treo khi động cơ hoạt động, có thể do sụt áp nguồn.
- Tín hiệu giữa tay cầm Tx và mạch thu Rx bị gián đoạn, sai chiều hoặc không ổn định.
- Drone bị rung lắc mạnh khi động cơ hoạt động ở tốc độ cao.
- Việc hiệu chỉnh thông số PID gặp khó khăn do cần nhiều lần thử nghiệm.

3.6. Kế hoạch kiểm thử

Bảng 7.1 (bên dưới, sẽ được đánh số lại thành bảng phù hợp trong bản hoàn chỉnh) trình bày kế hoạch kiểm thử tổng thể cho hệ thống, bao gồm các hạng mục cần kiểm tra, phương pháp thực hiện và kết quả mong đợi.

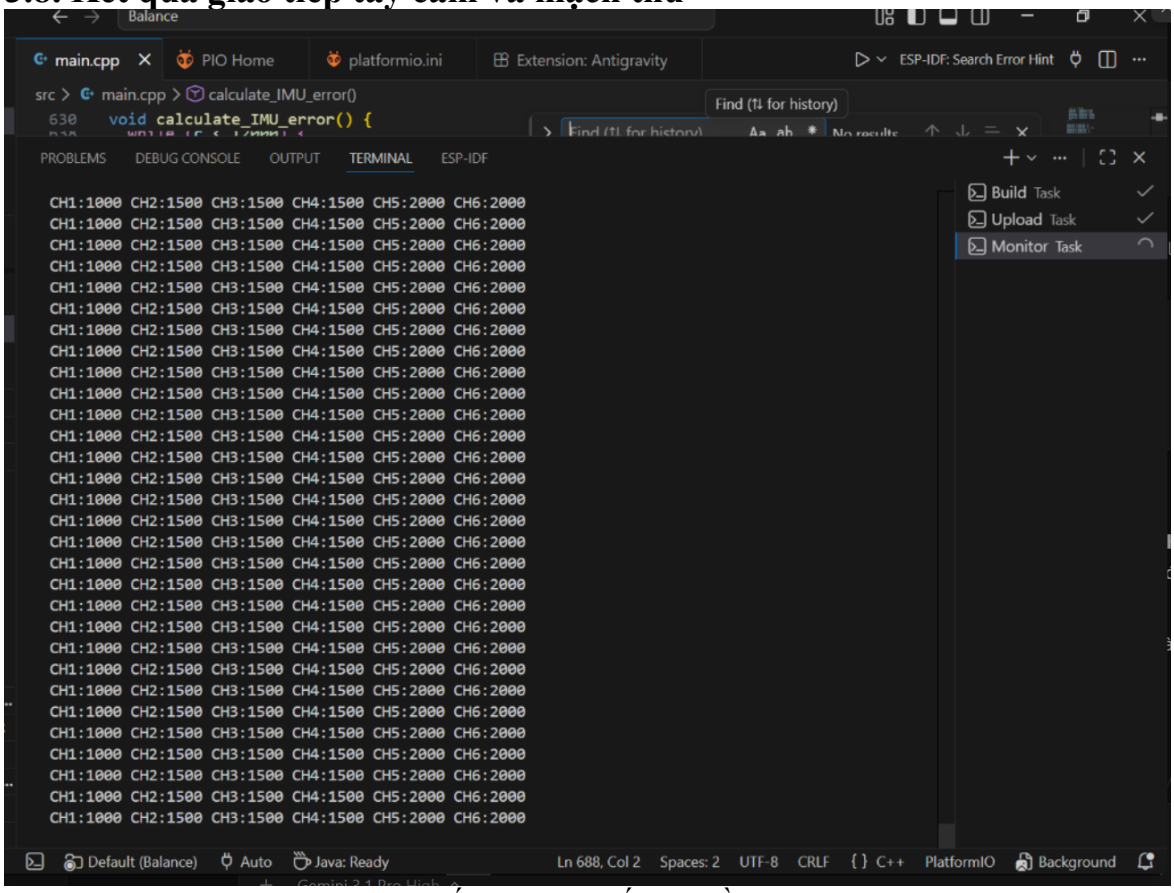
STT	Nội dung kiểm thử	Phương pháp	Kết quả mong đợi	Kết quả thực tế
1	ESP32 khởi động	Cấp nguồn, quan sát Serial 115200	In thông báo khởi động; không reset bất thường	Thành công
2	MPU6050 kết nối	Quét I²C GPIO21/22	Nhận địa chỉ 0x68; không dừng tại while(1)	Thành công
3	Dữ liệu IMU và Madgwick6DOF	Nghiêng cảm biến, in Acc/Gyro và roll/pitch/yaw	Roll/pitch đúng chiều; yaw có thể trôi do không có từ kế	Thành công
4	Liên kết Tx–Rx	Bật Microzone 6C Mini và MC7RE-V2	Mạch thu xuất đủ sáu kênh PWM ổn định	Thành công
5	Ánh xạ CH1–CH6	Thao tác từng cần/công tắc, in printRadioData()	CH1=Throttle, CH2=Roll, CH3=Pitch, CH4=Yaw, CH5=Throttle cut, CH6=Aux	Thành công
6	Hiệu chuẩn ESC	Bật calibrateESCs(), tháo cánh quạt	Bốn ESC nhận cùng lệnh throttle và hoàn tất hiệu chuẩn	Thành công
7	Đầu ra ESC	Tắt calibrateESCs(), kiểm tra GPIO26/33/14/27	Bốn ESC đáp ứng lệnh 0–180 tương ứng xung 1000–2000 µs	Thành công
8	PID và mixer	Nghiêng drone trên giá đỡ, quan sát printPIDoutput() và đầu ra	s1–s4 thay đổi đúng dấu theo roll/pitch/yaw	Thành công
9	Throttle cut và dữ liệu Rx lỗi	Thay đổi CH5; mô phỏng xung ngoài 800–2200 µs	Đầu ra về 0 khi throttle cut; kênh Rx về giá trị failsafe khi lỗi	Thành công
10	Thử nghiệm bay	Chỉ thử sau khi tắt chế độ calibrateESCs() và hoàn tất kiểm tra	Drone cất cánh và phản ứng cơ bản theo lệnh	Thất bại

3.7. Kết quả đọc MPU6050



Hình 17: Kết quả đọc MPU6050

3.8. Kết quả giao tiếp tay cầm và mạch thu



Hình 18: Kết quả giao tiếp tay cầm và mạch thu

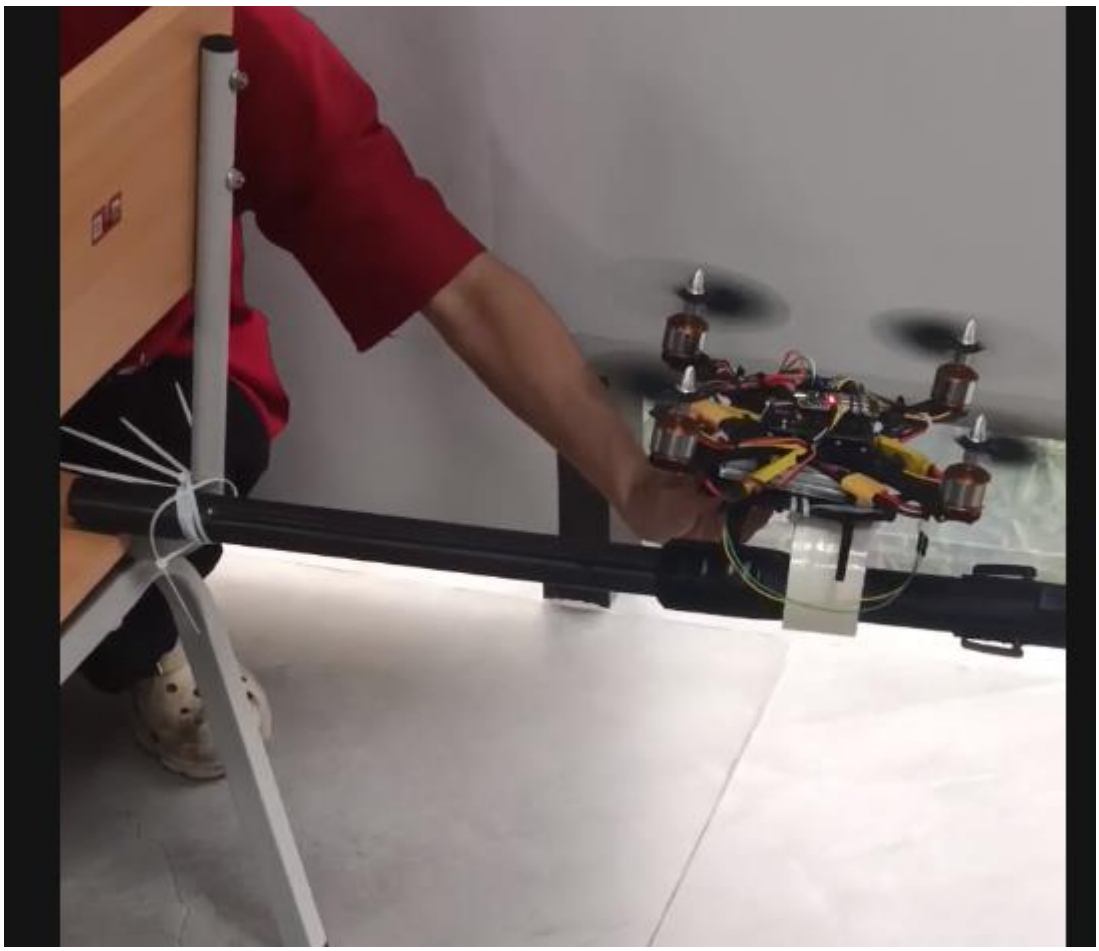
3.8. Kết quả cân bằng



Hình 19: Hình ảnh drone chưa được cân bằng

Tại thời điểm hoàn thành báo cáo, nhóm đã tiến hành thử nghiệm hệ thống nghiệm trên giá đỡ cố định (test stand tự chế sử dụng trục ống nhựa) cho phép drone nghiêng tự do theo một trục để đánh giá phản ứng bù ga của cảm biến và bộ điều khiển. Trong lần thử nghiệm này, hệ thống sử dụng bộ thông số thực nghiệm: roll/pitch với PID (0,2; 0,3; 0,05) và yaw rate PID (0,3; 0,05; 0,00015). Kết quả quan sát cho thấy khi ở mức ga thấp và có tác động nghiêng tay, các động cơ đã có phản ứng thay đổi tốc độ để bù trừ. Tuy nhiên, hệ thống xuất hiện hiện tượng rung lắc (oscillation) rất mạnh. Do drone đang được nẹp chặt trên trục ngang, hiện tượng trôi yaw (yaw drift) không thể hiện rõ ràng, nhưng hệ thống chống lại các tác động ngoại lực một cách thái quá và thiếu sự mượt mà. Kết luận hệ thống chưa đạt được trạng thái cân bằng tốt.

3.8. Kết quả bay thử



Hình 20: Hình ảnh drone khi bay thử

Phản ánh trung thực tình trạng thực tế của drone tại thời điểm thử nghiệm, hệ thống chưa đủ điều kiện an toàn để tiến hành cất cánh tự do.

Cụ thể, trong môi trường thử nghiệm trong nhà trên giá đỡ cố định, khi người điều khiển thực hiện thao tác tăng ga (throttle up), drone lập tức mất ổn định nghiêm trọng. Máy bay phản ứng cực kỳ gắt, mất kiểm soát hoàn toàn và ngay lập tức bị lật úp mạnh nhiều vòng quanh trục giá đỡ. Do đó, các thông số về độ cao và thời gian bay không thể ghi nhận. Mức độ ổn định của hệ thống hiện tại là không có. Nhóm cần phải đánh giá lại chiều quay động cơ, chiều cánh quạt hoặc tinh chỉnh lại (tuning) thông số PID (khả năng cao do thông số P đang quá lớn gây đáp ứng thái quá) trước khi có thể tháo khỏi giá đỡ để bay thử thực tế.

3.9. Đánh giá tổng thể

Dựa trên quá trình thiết kế và những kết quả kiểm thử thu được (sẽ được cập nhật đầy đủ ở các mục trên), hệ thống có thể được đánh giá sơ bộ theo các ưu điểm và hạn chế dự kiến như sau.

Ưu điểm dự kiến:

- Sử dụng tay cầm Microzone 6C Mini và mạch thu MC7RE-V2, thuận tiện cho thao tác trực tiếp các lệnh throttle, roll, pitch và yaw.
- Bộ điều khiển RF 2,4 GHz truyền lệnh trực tiếp đến mạch thu trong phạm vi hoạt động của hệ thống.
- Mạch thu MC7RE-V2 đưa sáu kênh xung PWM trực tiếp đến ESP32 và được đo bằng ngắt theo thời gian thực.
- Cấu hình động cơ brushless A2212 2450KV kết hợp ESC 30A và pin LiPo SHANG YI 3S 11,1 V – 2200 mAh phù hợp với khung F250 mà nhóm đã lựa chọn.
- Có MPU6050, bộ lọc thông thấp, thuật toán Madgwick và khả năng xử lý dữ liệu điều khiển theo thời gian thực.

Hạn chế dự kiến:

- Tín hiệu vô tuyến có thể bị ảnh hưởng bởi khoảng cách, nhiễu hoặc cấu hình chưa đúng giữa tay cầm Tx và mạch thu Rx.
- ESP32 DevKit có thể có khối lượng tương đối lớn so với mô hình drone mini, ảnh hưởng đến khả năng mang tải.
- Dữ liệu từ MPU6050 dễ bị ảnh hưởng bởi rung động cơ học từ động cơ.
- Việc hiệu chỉnh thông số PID đòi hỏi nhiều lần thử nghiệm thực tế và có thể mất nhiều thời gian.
- Thời lượng bay bị giới hạn bởi dung lượng 2200 mAh của pin và mức dòng tiêu thụ thực tế của bốn động cơ.
- Thiết kế khung và cách đi dây trong phiên bản đầu có thể chưa được tối ưu.
- Phiên bản mã nguồn được cung cấp đang bật calibrateESCs() trong setup(), nên phải tắt chế độ hiệu chuẩn này trước khi kiểm thử PID và bay.
- Hệ thống hiện tại chưa có khả năng giữ độ cao tự động hoặc định vị vị trí.

CHƯƠNG IV. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1. Kết luận

Trong quá trình thực hiện đồ án, nhóm đã nghiên cứu các kiến thức nền tảng liên quan đến drone quadcopter, bao gồm nguyên lý tạo lực nâng, các chuyển động throttle, roll, pitch, yaw, cũng như vi điều khiển ESP32, cảm biến MPU6050, bộ lọc thông thấp, thuật toán Madgwick, bộ điều khiển PID và hệ thống điều khiển vô tuyến.

Hệ thống dùng ESP32 làm trung tâm, đọc MPU6050 qua I²C, nhận sáu kênh PWM từ Microzone MC7RE-V2 và điều khiển bốn ESC trên GPIO26, GPIO33, GPIO14, GPIO27 bằng ESP32Servo. Mã nguồn tích hợp lọc thông thấp, Madgwick6DOF, PID, mixer, throttle cut và kiểm tra xung Rx. Nguồn chính là pin LiPo SHANG YI 3S 11,1 V – 2200 mAh; LM2596 tạo 5 V cho mạch điều khiển.

Những kết quả thực nghiệm cụ thể, bao gồm độ ổn định của MPU6050, tín hiệu tay cầm và mạch thu, khả năng điều khiển động cơ và mức độ cân bằng của drone, sẽ được nhóm bổ sung sau khi hoàn tất lắp ráp và kiểm thử.

Mã nguồn hiện tại đang gọi calibrateESCs() trong setup(); trạng thái này phù hợp khi hiệu chuẩn ESC nhưng phải được tắt để loop() điều khiển bay hoạt động.

Thông qua quá trình thực hiện đồ án, nhóm đã có cơ hội áp dụng kiến thức về hệ thống nhúng vào một sản phẩm thực tế có tính tích hợp cao giữa phần cứng và phần mềm, rèn luyện kỹ năng lập trình vi điều khiển, xử lý tín hiệu cảm biến, thiết kế mạch điện cơ bản và làm việc nhóm trong một dự án kỹ thuật có thời hạn.

4.2. Hướng phát triển

Trên cơ sở những kết quả đạt được (sẽ được cập nhật đầy đủ), nhóm đề xuất một số hướng phát triển tiếp theo cho đồ án nhằm nâng cao tính hoàn thiện và mở rộng chức năng của hệ thống trong tương lai:

- Thiết kế mạch PCB nhỏ gọn thay cho mạch lắp ráp thủ công hoặc board đục lỗ, giúp giảm khối lượng và tăng độ tin cậy của kết nối điện.
- Sử dụng bo mạch ESP32 phiên bản nhỏ gọn hơn nhằm giảm khối lượng tổng thể của drone.
- Thay thế MPU6050 bằng cảm biến IMU có độ chính xác cao hơn và ít bị ảnh hưởng bởi nhiễu rung động.

- Bổ sung cảm biến áp suất khí quyển (barometer) để hỗ trợ giữ độ cao tự động.
- Bổ sung chức năng giám sát điện áp pin theo thời gian thực và cảnh báo khi pin yếu.
- Nâng cấp tay cầm và mạch thu có nhiều kênh hơn hoặc hỗ trợ truyền dữ liệu phản hồi để tăng khả năng giám sát trạng thái drone.
- Tích hợp thêm camera để hỗ trợ quan sát hoặc truyền hình ảnh trực tiếp (FPV).
- Tích hợp module định vị GPS để hỗ trợ các chế độ bay tự động cơ bản.
- Tối ưu hóa các thông số PID thông qua quá trình thử nghiệm và hiệu chỉnh có hệ thống hơn.
- Nghiên cứu tối ưu tham số Madgwick hoặc so sánh với các phương pháp ước lượng tư thế khác như Kalman để nâng cao độ chính xác của góc nghiêng.
- Thiết kế lại khung drone theo hướng nhẹ hơn và cân đối hơn để cải thiện hiệu suất bay.

TÀI LIỆU THAM KHẢO

[1] Espressif Systems, "ESP32 Series Datasheet."
Đường dẫn:
https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.
Dùng để trích dẫn các thông số phần cứng, chân GPIO, giao tiếp I²C và khả năng tạo tín hiệu PWM của ESP32.

[2] Espressif Systems, "Arduino Core for ESP32 – Official Documentation."
Đường dẫn: https://docs.espressif.com/projects/arduino-esp32/en/latest/index.html.
Dùng để trích dẫn nội dung lập trình ESP32 bằng Arduino, sử dụng thư viện và cấu hình các ngoại vi.

[3] TDK InvenSense, "MPU-6000 and MPU-6050 Product Specification."
Đường dẫn: https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf.
Dùng để trích dẫn cấu tạo, nguyên lý hoạt động, dải đo và dữ liệu gia tốc kế, con quay hồi chuyển của MPU6050.

[4] NXP Semiconductors, "UM10204 – I²C-bus Specification and User Manual."
Đường dẫn: https://www.nxp.com/docs/en/user-guide/UM10204.pdf.
Dùng để trích dẫn nguyên lý giao tiếp I²C, vai trò của hai đường SDA, SCL và cơ chế truyền dữ liệu master–slave.

[5] Texas Instruments, "Pulse-Width Modulation Generator Overview."
Đường dẫn: https://www.ti.com/document-viewer/lit/html/SCLA082A.
Dùng để trích dẫn nguyên lý điều chế độ rộng xung PWM, chu kỳ, tần số và độ rộng xung điều khiển.

[6] MathWorks, "Proportional-Integral-Derivative (PID) Controllers."
Đường dẫn: https://www.mathworks.com/help/control/ug/proportional-integral-derivative-pid-controllers.html.
Dùng để trích dẫn nguyên lý bộ điều khiển PID và vai trò của các thành phần P, I, D trong quá trình cân bằng drone.

[7] PX4 Development Team, "ESCs & Motors – PX4 User Guide."
Đường dẫn: https://docs.px4.io/main/en/peripherals/esc_motors.
Dùng để trích dẫn nguyên lý hoạt động của động cơ brushless, chức năng của ESC và tín hiệu điều khiển động cơ trên drone.

[8] Video hướng dẫn được sử dụng trong quá trình tính PID. [How does PID controller work? | Simple Explaination on Quadcopter - YouTube] - Dùng để tính PID.